SUPPORT2 Dataset: Data Cleaning & Preparation

AAI-500 Applied Statistics for AI | Final Team Project

Notebook Objectives

This notebook covers the **Data Cleaning & Preparation** phase of the project pipeline:

1. Initial data inspection and profiling
2. Column renames per PEP-8
3. Engineering the binary target variable (`death_180d`)
4. Data type casting and ordinal/nominal encoding
5. Missing data analysis and imputation
6. Dropping unused columns
7. Documenting target / feature / outcome
8. Export of the cleaned dataset for EDA and modeling

Section 0: Setup & Imports

In [ ]:
```
import sys
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer

sys.path.append("../../utils")
from dataset import get_project_root, load_csv

warnings.filterwarnings("ignore")

current = Path.cwd().resolve()

while current != current.parent:
    if (current / ".git").exists() or (current / "pyproject.toml").exists():
        project_root = current
        break
    current = current.parent
```

```
PROJ_ROOT = get_project_root()
OUTPUT_PATH = PROJ_ROOT / "data" / "support2_cleaned.csv"
print(f"Output path  : {OUTPUT_PATH}")
```

Section 1: Load Data & Initial Inspection

In [ ]:
```
df_raw = load_csv("support2_raw_complete.csv")
print(f"Shape: {df_raw.shape[0]:,} rows  x  {df_raw.shape[1]} columns")
df_raw.head()
```

In [ ]:
```
df_raw.info(verbose=True, show_counts=True)
```

In [ ]:
```
int_cols = df_raw.select_dtypes(include="int64").columns
binary_cols = [
    c
    for c in int_cols
    if df_raw[c].nunique() == 2 and set(df_raw[c].unique()) == {0, 1}
]
print("===========Binary Columns==========")
print(binary_cols)
```

1.1 Descriptive Statistics

In [ ]:
```
desc = df_raw.describe().T
desc.columns = [
    "Count",
    "Mean",
    "Std Dev",
    "Min",
    "Q1 (25%)",
    "Median (50%)",
    "Q3 (75%)",
    "Max",
]
desc
```

Section 2: Column Renaming

Rename some columsn to follow PEP-8 standard

In [ ]:
```
rename_map = {
    "d.time": "d_time",
    "num.co": "num_co",
}
df = df_raw.rename(columns=rename_map).copy()

print("Column renames applied:")
```

```
for old, new in rename_map.items():
    print(f"  {old!r}  ->  {new!r}")
print(f"\nTotal columns: {len(df.columns)}")
```

Section 3: Target Variable

Did the patient die within 180 days of study?

To answer this we look at two fields:

- `death` : whether the patient ever died during 180 days (1 = yes, 0 = no)
- `d_time` : days of follow-up (how long they were observed). If this number is greater than 180 days of the trial, that means they survived the trial

death_180d (1 = yes, 0 = no) = {death=1 AND d_time <= 180}

Notice the AND conditional because the patient must've died within the 180 days to satisfy our target. We then use **death_180d** for binary classification

In [ ]:
```
df["death_180d"] = ((df["death"] == 1) & (df["d_time"] <= 180)).astype(int)

counts = df["death_180d"].value_counts()
rel_freqs = df["death_180d"].value_counts(normalize=True)

summary = pd.DataFrame(
    {
        "Outcome": ["Survived >= 180 days (0)", "Died within 180 days (1)"],
        "Frequency": counts.values,
        "Relative Frequency": rel_freqs,
        "Percentage (%)": (rel_freqs.values * 100),
    }
)
print(summary.to_string(index=False))
print(f"\nClass imbalance ratio (survived: died) = {counts[0] / counts[1]:.2
```

Section 4: Data Type Casting (into Pandas Type)

Correct data types are essential for proper computing and modeling. We apply three categories of correction. Casting to Int8 also handles NaN gracefully:

1. **Binary integer columns** -> `Int8` (nullable integer; memory-efficient)
2. **Nominal categorical columns** -> unordered `pd.Categorical`
3. **Ordinal categorical columns** -> ordered `pd.Categorical` for proper sorting and comparision

`income` ordering (ascending socioeconomic status): `under $11k` < `$11-$25k` < `$25-$50k` < `>$50k`

`sfdm2` ordering (best functional status → worst): `no disability` < `moderate ADL impairment` < `severe SIP impairment` < `coma/intubated` < `died before 2-month interview`

In [ ]:
```
# Binary columns -> nullable Int8
binary_cols = ["death", "hospdead", "diabetes", "dementia"]
for col in binary_cols:
    df[col] = df[col].astype("Int8")

######  8 Catagoricals
# sex = 2 levels
# dzgroup = 5
# dzclass = 4
# race = 5
# income = 4
# ca = 3
# dnr = 3
# sfdm2 = 5
##############

# Nominal (unordered) categoricals
nominal_cats = ["sex", "dzgroup", "dzclass", "race", "ca", "dnr"]
for col in nominal_cats:
    df[col] = df[col].astype("category")

# Ordinal: income (lowest -> highest bracket) as coded in support2_data_dict
income_order = ["under $11k", "$11-$25k", "$25-$50k", ">$50k"]
df["income"] = pd.Categorical(df["income"], categories=income_order, orderec

# Ordinal: sfdm2 (2-month functional disability; no disability -> died befor
# as coded in support2_data_dictionary.md
sfdm2_order = [
    "no(M2 and SIP pres)",  # Level 1 - no moderate/severe disability
    "adl>=4 (>=5 if sur)",  # Level 2 - unable to do 4+ ADL
    "SIP>=30",  # Level 3 - Sickness Impact Profile >= 30
    "Coma or Intub",  # Level 4 - intubated or in coma at 2 months
    "<2 mo. follow-up",  # Level 5 - died before 2-month interview
]
df["sfdm2"] = pd.Categorical(df["sfdm2"], categories=sfdm2_order, ordered=Tr

print("Data type corrections applied:\n")
cols_updated = binary_cols + nominal_cats + ["income", "sfdm2"]
print(df[cols_updated].dtypes.to_string())
```

Section 5: Missing Data Analysis

The authors have stated there are a maximum of 5641 N/As. This, along with every other missing data, we must verify and address.

In [ ]:
```
missing_count = df.isnull().sum()
missing_percent = missing_count / len(df) * 100
missing_df = pd.DataFrame(
    {
        "Missing Count": missing_count,
        "Missing %": missing_percent,
    }
).sort_values("Missing Count", ascending=False)

missing_df = missing_df[missing_df["Missing Count"] > 0]

total_cells = df.shape[0] * df.shape[1]
total_missing = int(missing_df["Missing Count"].sum())

print(f"Columns with at least one missing value : {len(missing_df)}")
print(
    f"Total missing cells : {total_missing:,} / {total_cells:,}  ({total_mis
)
print(missing_df.to_string())

# verify maximum missing n/as
max_na = df_raw.isnull().sum().max()
max_na_col = df_raw.isnull().sum().idxmax()
print(f"Max NAs: {max_na} (column: '{max_na_col}')")
assert max_na == 5641, f"Expected 5641, got {max_na}"
```

In [ ]:
```
fig, ax = plt.subplots(figsize=(11, 6))

bars = ax.barh(
    missing_df.index[::-1],
    missing_df["Missing %"][::-1],
    edgecolor="white",
    height=0.7,
)


ax.set_xlabel("Missing Data (%)", fontsize=11)
ax.set_title("Missing Data Rate by Column", fontsize=13, fontweight="bold")
ax.set_xlim(0, 80)

for bar, pct in zip(bars, missing_df["Missing %"][::-1]):
    if pct > 0:
        ax.text(
            bar.get_width() + 0.5,
            bar.get_y() + bar.get_height() / 2,
            f"{pct:.1f}%",
            va="center",
            fontsize=8,
        )
plt.show()
```

5.1 Missing Data Decision Table

Here we decide how to tackle missing data. Either drop the column entirely or impute mathematically

For most of our biological features that have missing data, we compute the median. The reason is per Agresti et al (2022), chapter 3, that for right-skewed data, using median instead of mean worse the skewness of the data (push it woard the tail). Looking at section 1, our columns the mean > median, which signals right skewed.

Column(s) Decision Rationale

| `adlp`, `adls` | **Drop** | Superseded by `adlsc` per original authors | `dnrday` | **Drop** | Do-no-resus day. Useless for our purpose | `hday` | **Drop** | Day admitted to study. Nothing to do with mortality | `charges, totcst, totmcst` | **Drop** | Relating to hospital cost, which we won't be studying | `alb`, `pafi`, `bili`, `crea`, `bun`, `wblc`, `urine` | **imputed values avail** | Per Author's note we have the recommended imputed values | `ph`, `glucose`, `scoma`, `avtisst`, `edu`, `prg2m`, `prg6m` | **Sample median** | high missing | | `surv2m`, `surv6m`, `aps`, | **Sample median** | kept as benchmark columns | | `meanbp`, `hrt`, `resp`, `temp`, `sod` | **Sample median** | few missing | | `charges`, `totcst` | **Sample median** | Retained for future cost analysis | | `totmcst` | **Sample median** | Retained for future cost analysis; many missing values and will be difficult to impute. Contains negative values (balances) | | `race`, `income`, `sfdm2`, `dnr` | **Mode imputation** | Most frequent category as best single-value estimate for categoricals |

Benchmark Columns (kept but will be excluded from death_180d model features)

Column	Role	Notes
`surv2m`, `surv6m`	SUPPORT model predictions	will be compared against our model
`aps`	APACHE III severity score	Independent benchmark system widely used in critical care
`sps`	SUPPORT physiology sub-score	Component of the SUPPORT model ??
`prg2m`, `prg6m`	Physician survival estimates	ML vs. clinician judgment comparison
`dnr`	DNR order status	**Should not use as predictor feature as it's reflects human behavior rather than biological factors**

Section 6: Feature Pruning

We remove features not used in our analysis:

In [ ]:
```
cols_to_drop = ["adlp", "adls", "dnrday"]

df.drop(columns=cols_to_drop, inplace=True, errors="ignore")
print(f"Dropped   : {len(cols_to_drop)} columns")
print(f"Remaining: {df.shape[0]:,} rows  x  {df.shape[1]} columns")
print(f"Remaining columns ({df.shape[1]}):")
print(list(df.columns))
```

Section 6.5: Zero-Value Treatment

Many physiologic measurements in SUPPORT2 use `0` as a sentinel for a missing or invalid reading rather than a true value. A living patient cannot have a mean blood pressure, heart rate, respiratory rate, white-blood-cell count, or serum glucose of exactly zero. We therefore treat zeros in these columns as missing (`NaN`) **before imputation (Section 7)**, so each column is filled by the same clinically validated strategy already chosen for it (domain normal-fill for labs, sample median for vitals) instead of being left as a physiologically impossible value. Computing the median *after* removing these zeros also produces a more representative fill value, since the spurious zeros no longer drag the center downward.

Columns where `0` is a legitimate value (left untouched)

Column(s)	Why `0` is valid
`death` , `hospdead` , `diabetes` , `dementia`	Binary indicators (0 = no)
`num_co`	Count of comorbidities (0 = none)
`scoma`	SUPPORT coma score (0 = fully alert)
`adlsc` , `adlp` , `adls`	ADL impairment scores (0 = no impairment)
`edu`	Years of education (0 = no formal schooling)
`aps` , `sps`	Severity scores (0 = minimal physiologic derangement)
`surv2m` , `surv6m` , `prg2m` , `prg6m`	Survival probabilities / estimates (0 = predicted death)

Column(s)	Why `0` is valid
`charges`, `totcst`, `totmcst`	Costs (0 / negative = billing adjustments)
`death_180d`	Engineered binary target

Physiologic columns where `0` is invalid → converted to `NaN`

`meanbp`, `hrt`, `resp`, `temp`, `sod`, `ph`, `glucose`, `wblc`, `alb`, `bili`, `crea`, `bun`, `pafi`, `urine`.

Clinical caveat: `urine`: sustained complete anuria (`urine = 0`) can occur in acute renal failure, but a recorded *daily* output of exactly zero is far more often a not-recorded artifact. We treat it as missing and apply the SUPPORT normal-fill value, consistent with the original authors' imputation convention.

Motivating reasoning: rows carrying a zero vital sign show a markedly higher death rate than the cohort average, and the zeros are spread across several variables rather than concentrated in one. This pattern is consistent with mixed measurement/recording artifacts in very sick patients rather than true readings, so imputing them (rather than dropping the rows) preserves sample size without fabricating implausible values.

In [ ]:
```
# Columns where 0 is a legitimate value -- leave untouched
ZERO_ALLOWED = [
    "death",
    "hospdead",
    "diabetes",
    "dementia",  # binary indicators
    "num_co",  # comorbidity count
    "scoma",  # coma score (0 = alert)
    "adlsc",
    "adlp",
    "adls",  # ADL scores (0 = no impairment)
    "edu",  # years of education
    "aps",
    "sps",  # severity scores
    "surv2m",
    "surv6m",
    "prg2m",
    "prg6m",  # survival probs / estimates
    "charges",
    "totcst",
    "totmcst",  # costs (0/negative = billing)
    "death_180d",  # engineered target
]

ZERO_INVALID = [
    "meanbp",
```

```
    "hrt",
    "resp",
    "temp",
    "sod",
    "ph",
    "glucose",
    "wblc",
    "alb",
    "bili",
    "crea",
    "bun",
    "pafi",
    "urine",
]

# --- Scan: every numeric column that currently holds a zero ---
zero_log = []
for col in df.select_dtypes(include=["number"]).columns:
    n_zero = int((df[col] == 0).sum())
    if n_zero == 0:
        continue
    if col in ZERO_INVALID:
        treatment = "-> NaN"
    elif col in ZERO_ALLOWED:
        treatment = "allowed"
    else:
        treatment = "UNCLASSIFIED"
    zero_log.append(
        {
            "Column": col,
            "Zeros": n_zero,
            "Zero %": round(n_zero / len(df) * 100, 2),
            "Treatment": treatment,
        }
    )

zero_scan = pd.DataFrame(zero_log).sort_values("Zeros", ascending=False)
print("Zero-value scan (numeric columns containing at least one zero):\n")
print(zero_scan.to_string(index=False))

# Guard: any numeric column with zeros must be explicitly classified so new
# columns can never silently slip through this treatment step.
unclassified = zero_scan.loc[
    zero_scan["Treatment"] == "UNCLASSIFIED", "Column"
].tolist()
assert not unclassified, f"Unclassified zero columns: {unclassified}"

total_converted = 0
for col in ZERO_INVALID:
    if col not in df.columns:
        continue
    n_zero = int((df[col] == 0).sum())
    if n_zero > 0:
        df[col] = df[col].replace(0, np.nan)
        total_converted += n_zero
        print(f"  {col:<8}: {n_zero:>3} zeros -> NaN")
```

```
print(f"\nTotal zeros converted to NaN: {total_converted}")
assert int(df.shape[0]) == 9105 and int(df.shape[1]) == 46
print(f"Dataset shape unchanged: {df.shape[0]:,} rows x {df.shape[1]} columr
```

Section 7: Missing Data Imputation

The original SUPPORT study authors (Harrell, 2022) provide clinically validated **normal fill-in values** for seven physiological variables (see docs/support2_dataset_description.md). We will use this for imputation

Variable	Normal Fill	Meaning
`alb`	3.5	Serum albumin - liver / nutrition
`pafi`	333.3	PaO2/FiO2 ratio - oxygen level
`bili`	1.01	Bilirubin - liver function
`crea`	1.01	Creatinine - renal function
`bun`	6.51	BUN - normal renal function
`wblc`	9.0	WBC count - immune status
`urine`	2502	Urine output - renal output

In [ ]:
```
DOMAIN_FILL = {
    "pafi": 333.3,
    "bili": 1.01,
    "crea": 1.01,
    "bun": 6.51,
    "wblc": 9.0,
    "urine": 2502.0,
}

# KNN imputation for albumin
# Rationale: albumin has 37% missingness and is a clinically important morta
# predictor (Knaus et al., 1995). The domain fill value of 3.5 coincides wit
# the Q3 boundary, collapsing quartile bins and making albumin unusable as a
# stratification variable in interaction analysis. KNN imputation (k=5) pres
# realistic variation by borrowing from physiologically similar patients rat
# than substituting a single normal value uniformly across all missing cases
# Features used for neighbor calculation: age, meanbp, hrt, resp, temp, sod;
# these are variables with low missingness that capture overall physiologic

alb_features = ["alb", "age", "meanbp", "hrt", "resp", "temp", "sod"]
knn_imputer = KNNImputer(n_neighbors=5)

df_alb = df[alb_features].copy()
df_alb_imputed = pd.DataFrame(
    knn_imputer.fit_transform(df_alb), columns=alb_features, index=df.index
)
```

```
df["alb"] = df_alb_imputed["alb"]

# Verify
print(f"Albumin missing after KNN imputation: {df['alb'].isnull().sum()}")
print(df["alb"].describe().round(3))

imputation_log = []

for k, v in DOMAIN_FILL.items():
    n_missing = int(df[k].isnull().sum())
    if n_missing > 0:
        df[k] = df[k].fillna(v)
        imputation_log.append(
            {
                "Column": k,
                "Strategy": "Domain Fill",
                "Fill Value": v,
                "Cells Filled": n_missing,
            }
        )

MEDIAN_COLS = [
    "sps",
    "charges",
    "totcst",
    "totmcst",
    "meanbp",
    "hrt",
    "resp",
    "temp",
    "sod",
    "ph",
    "glucose",
    "scoma",
    "avtisst",
    "edu",
    "surv2m",
    "surv6m",
    "aps",
    "prg2m",
    "prg6m",
]
for k in MEDIAN_COLS:
    n_missing = int(df[k].isnull().sum())
    if n_missing > 0:
        med = float(df[k].median())
        df[k] = df[k].fillna(med)
        imputation_log.append(
            {
                "Column": k,
                "Strategy": "Sample Median",
                "Fill Value": round(med, 4),
                "Cells Filled": n_missing,
            }
        )
```

```
MODE_COLS = ["race", "income", "sfdm2", "dnr"]

for k in MODE_COLS:
    n_missing = int(df[k].isnull().sum())
    if n_missing > 0:
        mode_val = df[k].mode()[0]
        df[k] = df[k].fillna(mode_val)
        imputation_log.append(
            {
                "Column": k,
                "Strategy": "Sample Mode",
                "Fill Value": str(mode_val),
                "Cells Filled": n_missing,
            }
        )

log_df = pd.DataFrame(imputation_log)
print("Imputation Summary:")
print(log_df.to_string(index=False))
print(f"Total cells imputed: {log_df.get('Cells Filled', np.array([])).sum()
```

In [ ]:
```
remaining_na = df.isnull().sum()
remaining_na = remaining_na[remaining_na > 0]

if len(remaining_na) == 0:
    print("Imputation complete, zero missing values remain in any column.")
else:
    print("WARNING: Remaining missing values:")
    print(remaining_na.to_string())

print(
    f"\nDataset shape after imputation: {df.shape[0]:,} rows  x  {df.shape[1
)
```

In [ ]:
```
# ── Section 7.1: Physiologic Range Validation (Winsorization) ──────────
#
# Invalid sentinel zeros were converted to NaN in Section 6.5 and filled dur
# imputation, so no physiologically impossible zeros remain in the vitals/la
# What remains to handle are implausible *non-zero* extremes carried in the
# data. Rather than dropping these rows, we winsorize: values outside establ
# physiologic reference ranges are capped at the boundary, preserving sample
# while eliminating impossible values.
#
# References:
# Knaus et al. (1991). The APACHE III prognostic system. Chest, 100(6), 1619
# Knaus et al. (1995). The SUPPORT prognostic model. Ann Intern Med, 122(3),

phys_limits = {
    "meanbp": (1, 200),  # mmHg
    "hrt": (1, 250),  # bpm
    "resp": (1, 60),  # breaths/min
    "temp": (25, 42),  # Celsius
    "sod": (100, 170),  # mEq/L
    "glucose": (20, 600),  # mg/dL
    "bun": (0, 150),  # mg/dL
```

```
    "wblc": (0, 50),  # thousands/uL
    "pafi": (50, 600),  # ratio
    "urine": (0, 8000),  # mL/day
}

print("Winsorization summary:")
total_capped = 0
for col, (lo, hi) in phys_limits.items():
    if col not in df.columns:
        continue
    n_capped = int(((df[col] < lo) | (df[col] > hi)).sum())
    if n_capped > 0:
        df[col] = df[col].clip(lower=lo, upper=hi)
        print(f"  {col}: {n_capped} values capped to [{lo}, {hi}]")
        total_capped += n_capped

print(f"\nTotal values winsorized: {total_capped}")
print(f"Final dataset shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
```

In [ ]:
```
# Verify target variable balance after row drops
counts = df["death_180d"].value_counts()
print("\nUpdated class balance:")
print(f"Survived >= 180 days: {counts[0]:,} ({counts[0] / len(df):.1%})")
print(f"Died within 180 days: {counts[1]:,} ({counts[1] / len(df):.1%})")
```

Section 8: Defining Feature, Outcome Separation, Leakage

Category	Columns
Target	`death_180d`
Outcome columns (excluded due to result leakage)	`death` , `hospdead` , `d_time` , `slos`
Row identifier	`id`
Features	All remaining columns

Section 9: Post-Cleaning Summary

We compare the dataset before and after cleaning and present the five-number summary for key physiological lab variables to confirm that zero-value treatment, imputation, and outlier winsorization produced a coherent, analysis-ready dataset. The cleaning-actions block quantifies each step: invalid physiologic zeros converted to `NaN` , cells imputed, and values winsorized.

In [ ]:
```
print("=== Pre-Cleaning ===")
print(f"  Shape          : {df_raw.shape[0]:,} rows  x  {df_raw.shape[1]} co
print(f"  Missing cells  : {df_raw.isnull().sum().sum():,}")

raw_invalid_zeros = int(
    sum((df_raw[c] == 0).sum() for c in ZERO_INVALID if c in df_raw.columns)
)

print("\n=== Cleaning Actions ===")
print(
    f"  Invalid zeros treated : {raw_invalid_zeros:,} (converted to NaN, the
)
print(f"  Cells imputed         : {int(log_df['Cells Filled'].sum()):,}")
print(f"  Values winsorized     : {total_capped:,}")

print("\n=== Post-Cleaning ===")
print(f"  Shape          : {df.shape[0]:,} rows  x  {df.shape[1]} columns")
print(f"  Missing cells  : {df.isnull().sum().sum()}")
print(
    f"  Invalid zeros  : {int(sum((df[c] == 0).sum() for c in ZERO_INVALID i
)
print(f"  Rows retained  : {df.shape[0] / df_raw.shape[0] * 100:.1f}%")

prev = df["death_180d"].mean()
```

9.1 Important Lab Variables Summary (Post-Cleaning)

In [ ]:
```
key_labs = [
    "age",
    "meanbp",
    "hrt",
    "temp",
    "alb",
    "bili",
    "crea",
    "glucose",
    "bun",
    "urine",
]
post_desc = df[key_labs].describe().T
post_desc.columns = ["Count", "Mean", "Std Dev", "Min", "Q1", "Median", "Q3"
post_desc[["Min", "Q1", "Median", "Q3", "Max", "Mean", "Std Dev"]]
```

9.2 Target Variable Validation

The `death_180d` target is valid only if `d_time` is measured from study admission consistently across all patients. Two checks were performed to confirm this.

Check 1 — Admission reference point: For all 2,360 patients who died in hospital (`hospdead = 1`), `d_time` minus `slos` (length of stay) equals exactly

zero across every patient (mean = 0.0, std = 0.0, max = 0.0). This confirms `d_time` is admission-referenced with no exceptions.

Check 2 — Target variable integrity: All 4,265 patients classified as `death_180d = 1` have `d_time <= 180` (max = 180). No patient is incorrectly flagged. The median `d_time` of 23 days among this group indicates that most 180-day deaths in this dataset are early and acute rather than gradual.

Both checks pass. The `death_180d` target variable is correctly constructed and suitable for modeling.

In [ ]:
```
# Patients who died in hospital should have d_time = slos (length of stay)
# or very close to it
hospdead_patients = df[df["hospdead"] == 1]
assert (hospdead_patients["d_time"] == hospdead_patients["slos"]).all(), (
    "d_time mismatch detected"
)
print("Check passed: d_time == slos for all in-hospital deaths")
```

In [ ]:
```
# d_time for death_180d=1 patients should all be <= 180
died_180 = df[df["death_180d"] == 1]
print(died_180["d_time"].describe().to_string())
```

Section 10: Save Cleaned Dataset

In [ ]:
```
df.to_csv(OUTPUT_PATH, index=False)
print(f"Cleaned dataset saved to : {OUTPUT_PATH}")
print(f"Final shape              : {df.shape[0]:,} rows  x  {df.shape[1]} co
print("\nColumn list and dtypes:")
print(df.dtypes.to_string())
```

Imputation Bias Check

Several laboratory columns carried substantial missingness before imputation (glucose, albumin, and bilirubin roughly 25 to 49 percent), while creatinine and white-cell count were nearly complete. This check examines whether that missingness was related to the outcome, which is the missing-at-random assumption the downstream models depend on. The raw table is read only to recover the original missing positions, then mortality is compared between originally-missing and originally-present rows.

In [ ]:
```
# imputation bias check (moved here from EDA per PR #9 review)
# read raw only to recover the original missing positions, then compare mort
raw = df_raw  # reuse the pristine raw already loaded above (PR #9: no redur
dtime = "d.time" if "d.time" in raw.columns else "d_time"
y_raw = ((raw["death"] == 1) & (raw[dtime] <= 180)).astype(int)
```

```
report = []
for col in ["alb", "bili", "pafi", "glucose", "wblc", "crea"]:
    if col not in raw.columns:
        continue
    missing = raw[
        col
    ].isna()  # rows where this lab was originally missing (pre-imputation)
    report.append(
        {
            "variable": col,
            "pct_imputed": round(missing.mean() * 100, 1),
            # gap = mortality of originally-missing minus originally-present
            # near 0 means missingness is unrelated to the outcome (supports
            "gap": round(y_raw[missing].mean() - y_raw[~missing].mean(), 2),
        }
    )
print(
    pd.DataFrame(report)
    .sort_values("pct_imputed", ascending=False)
    .to_string(index=False)
)
```

The mortality gap between originally-missing and originally-present rows stayed within 0.05 for every column examined (the largest, creatinine at 0.05, rests on only 0.7 percent missingness and is effectively noise). The heavily-imputed columns (glucose, albumin, bilirubin) show no meaningful link between being missing and the outcome, so the normal-value fill does not appear to bias the outcome association. This supports modeling on the imputed columns and provides the missing-data note for the Limitations section.

SUPPORT2 Dataset: Exploratory Data Analysis

AAI-500 Applied Statistics for AI | Final Team Project

Notebook Objectives

This notebook covers the **Exploratory Data Analysis** phase of the project pipeline:

1. Detect Outliers
2. Univariate Analysis
3. Bivariate Analsysis
4. Demographic Analsysis
5. Disease Group Analysis

6. DNR Status Analysis

Learnings from the 1990s study

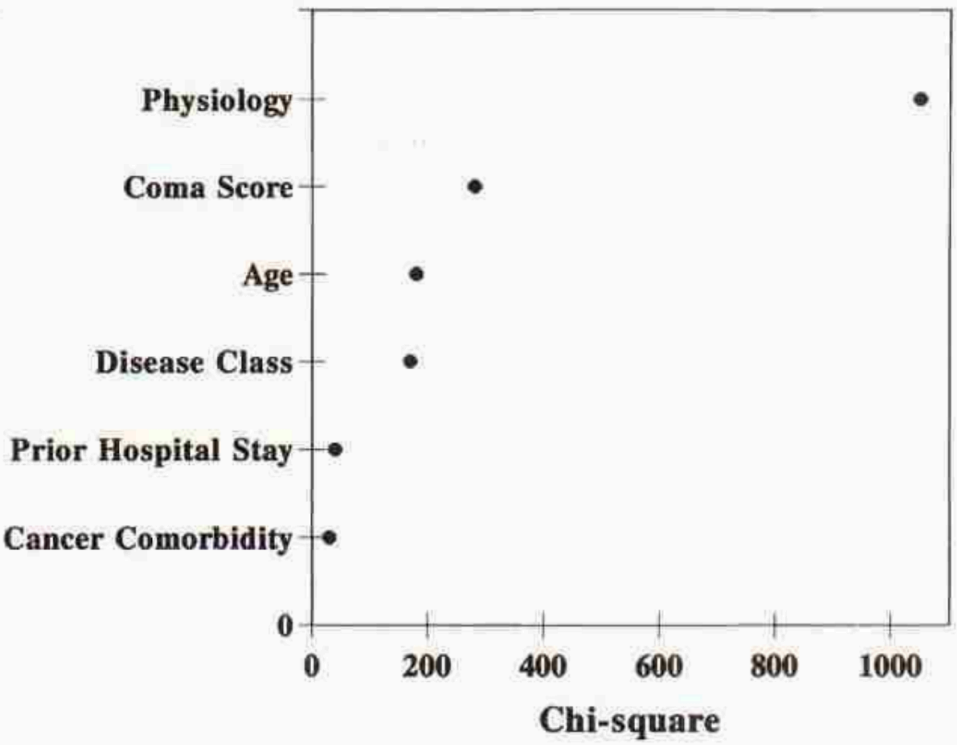

Figure 1. Relative contributions of major prognostic elements in the SUPPORT model as measured by the amount of chi-square accounted for by each element. Physiology = all physiologic measures except Glasgow coma scale; cancer comorbidity = presence of cancer in addition to disease category; previous hospital stay = days in the hospital before study entry.

To inform our work, we review Knaus et al's (1995) paper. According to Knaus et al these features were most weighted in their model:

- Physiology Score (calculated from wbc, albumin, etc). Some salient ones are Blood Pressure and Oxygenation levels
- Neurological Function (Glasgow coma scale)
- Patient age (but only when coupled with certain diseases like COPD)
- Apache III score
- Disease-Specific Interactions: The mathematical weight of certain features was explicitly programmed to vary based on the patient's primary

disease:Albumin x Disease: A low albumin level strongly predicted death in cancer, COPD, and congestive heart failure, but was relatively unimportant in coma or MOSF. WBC x Disease: A low WBC count was specifically associated with a greater risk of death in acute respiratory failure and MOSF. Age x Disease: Advancing age heavily impacted short-term survival in COPD but had almost no incremental effect on patients with severe acute complications like MOSF or malignancy.

Section 0: Setup & Imports

In [ ]:
```
import sys
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from scipy import stats
from sklearn.metrics import roc_curve, roc_auc_score

_here = Path.cwd()
_proj_root = _here if (_here / "utils").exists() else (_here / "../..").resc
if str(_proj_root) not in sys.path:
    sys.path.insert(0, str(_proj_root))

from utils.dataset import load_csv  # noqa: E402

warnings.filterwarnings("ignore")
```

Section 1: Load Cleaned Data

In [ ]:
```
df = load_csv("support2_cleaned.csv")
print(f"Shape: {df.shape[0]:,} rows  x  {df.shape[1]} columns")

OUTCOME_COLS = ["death", "hospdead", "d_time", "slos"]
ID_COLS = ["id"]
TARGET_COL = "death_180d"
BENCHMARK_COLS = ["surv2m", "surv6m", "aps", "sps", "prg2m", "prg6m", "dnr"]
LAB_COLS = [
    "meanbp",
    "wblc",
    "hrt",
    "resp",
    "temp",
    "pafi",
    "alb",
    "bili",
```

```
    "crea",
    "sod",
    "ph",
    "glucose",
    "bun",
    "urine",
]
FEATURE_COLS = [
    c
    for c in df.columns
    if c not in OUTCOME_COLS + ID_COLS + [TARGET_COL] + BENCHMARK_COLS
]

survivors = df[df[TARGET_COL] == 0]
died = df[df[TARGET_COL] == 1]
overall_rate = df[TARGET_COL].mean()
print(f"Survived 180d   : {len(survivors):,}  ({1 - overall_rate:.1%})")
print(f"Died within 180d: {len(died):,}  ({overall_rate:.1%})")
```

Section 2: Physiology Outlier Detection (IQR Method)

```
IQR = Q3 - Q1
Lo whisker = Q1 - 1.5IQR
Hi whisker = Q3 + 1.5IQR
```

- Urine boxplot median is pushed all the way right because of imputation with Norm-Fill.
- Glucose boxplot collapses due to high number of imputed NAs.
- **Probably better to run outliers before imputation**

In [ ]:
```
def compute_iqr_outliers(df, cols):
    rows = []
    for col in cols:
        Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
        IQR = Q3 - Q1
        lo, hi = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
        outliers = int(((df[col] < lo) | (df[col] > hi)).sum())
        rows.append(
            {
                "Column": col,
                "Q1": round(float(Q1), 3),
                "Q3": round(float(Q3), 3),
                "IQR": round(float(IQR), 3),
                "Lo Whisker": round(float(lo), 3),
                "Hi Whisker": round(float(hi), 3),
                "Outliers": outliers,
                "Outlier %": round(outliers / len(df) * 100, 1),
            }
        )
```

```
    return pd.DataFrame(rows).sort_values("Outlier %", ascending=False)


outlier_df = compute_iqr_outliers(df, LAB_COLS)
print(
    outlier_df[
        ["Column", "Lo Whisker", "Hi Whisker", "Outliers", "Outlier %"]
    ].to_string(index=False)
)
```

In [ ]:
```
top6 = outlier_df.head(6)["Column"].tolist()
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for i, col in enumerate(top6):
    axes.flat[i].boxplot(
        df[col],
        vert=False,
        medianprops=dict(color="red", linewidth=2),
        flierprops=dict(marker="o", alpha=0.2, markersize=3, color="gray"),
    )
    axes.flat[i].set_title(col)
    axes.flat[i].set_xlabel("Value")
fig.suptitle("Top 6 High Outliers Lab Variables", fontsize=12, fontweight="b
plt.tight_layout()
plt.show()
```

Section 3: Univariate Distributions

- Lab features are mainly right skewed
- Some imputations cause spikeness (glucose, pafi, bili)

In [ ]:
```
fig, axes = plt.subplots(4, 4, figsize=(30, 25))
for i, col in enumerate(LAB_COLS):
    axes.flat[i].hist(df[col], bins=50, edgecolor="white")
    axes.flat[i].set_title(col)
    axes.flat[i].set_xlabel("Value")
    axes.flat[i].set_ylabel("Count")
for j in range(len(LAB_COLS), len(axes.flat)):
    axes.flat[j].set_visible(False)
fig.suptitle(
    "Univariate Distributions - Day-3 Lab Variables", fontsize=13, fontweigh
)
plt.tight_layout()
plt.show()
```

3.1 Age follows normal distribution

In [ ]:
```
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(df["age"], bins=40, edgecolor="white")
```

```
ax.set_title("Age Distribution")
plt.show()
```

Section 4: Bivariate Analysis - Lab Values by Survival Outcome

- Difficult to see any impact these lab values make with this analysis
- The paper did say this only becomes evident when combined with Disease group. For example, a high creatine with kidney failure gives high likelihood of mortality.

In [ ]:
```
fig, axes = plt.subplots(4, 4, figsize=(16, 14))
for i, col in enumerate(LAB_COLS):
    ax = axes.flat[i]
    sns.boxplot(
        data=df,
        y=TARGET_COL,
        x=col,
        ax=ax,
        orient="h",
        palette=["lightgreen", "pink"],
        width=0.5,
        flierprops=dict(marker=".", alpha=0.2, markersize=3),
    )
    ax.set_title(col, fontsize=10)
    ax.set_xlabel("Value")
    ax.set_ylabel("")
    ax.set_yticks([0, 1])
    ax.set_yticklabels(["Survived", "Died"])

for j in range(len(LAB_COLS), len(axes.flat)):
    axes.flat[j].set_visible(False)

fig.suptitle(
    "Lab Values by Survival Outcome",
    fontsize=12,
    fontweight="bold",
)
plt.tight_layout()
plt.show()
```

Section 5: Demographic Analysis

- Mortality gradual decline as wealth increases
- Asian group a bit more mortality

In [ ]:

```
def plot_mortality_rate(df, col, target_col, overall_rate, ax, title=None, s
    rate = df.groupby(col, observed=True)[target_col].mean()
    rate = rate.sort_values(ascending=False)
    bars = ax.bar(
        range(len(rate)),
        rate.values * 100,
        edgecolor="white",
        width=0.6,
    )
    ax.axhline(
        overall_rate * 100,
        color="black",
        linestyle="--",
        linewidth=1.2,
        label=f"Overall ({overall_rate:.1%})",
    )
    ax.set_title(title or f"Mortality Rate by {col}")
    ax.set_ylabel("180-Day Mortality (%)")
    ax.set_xticks(range(len(rate)))
    ax.set_xticklabels(rate.index, rotation=30, ha="right", fontsize=8)
    ax.legend(fontsize=8)
    for bar, v in zip(bars, rate.values):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.5,
            f"{v:.1%}",
            ha="center",
            fontsize=8,
        )
```

```
# education bins
edu_bins = [0, 8, 12, 15, 100]
edu_labels = ["0-8 yrs", "9-12 yrs", "13-15 yrs", "16+ yrs"]
df["edu_group"] = pd.cut(df["edu"], bins=edu_bins,
labels=edu_labels, right=True)

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

plot_mortality_rate(
    df, "edu_group", TARGET_COL, overall_rate, axes[0, 0],
title="Education", sort=False
)
plot_mortality_rate(df, "income", TARGET_COL, overall_rate, axes[0,
1], title="Income")
plot_mortality_rate(df, "race", TARGET_COL, overall_rate, axes[0,
2], title="Race")
plot_mortality_rate(df, "sex", TARGET_COL, overall_rate, axes[1,
0], title="Sex")
plot_mortality_rate(
    df, "ca", TARGET_COL, overall_rate, axes[1, 1], title="Cancer
Status"
)
axes[1, 2].set_visible(False)
```

```
fig.suptitle(
    "Section 5: 180-Day Mortality Rate by Demographic & Categorical
Features",
    fontsize=12,
    fontweight="bold",
)
plt.tight_layout()
plt.show()

# Survivorship by age
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.boxplot(data=df, x=TARGET_COL, y="age", ax=axes[0], palette=
["green", "red"])
axes[0].set_title("Age by Survival Outcome")
axes[0].set_xticklabels(["Survived", "Died"])
axes[0].set_xlabel("")
axes[0].set_ylabel("Age (years)")

# KDE overlay
for label, grp, color in [("Survived", survivors, "green"),
("Died", died, "red")]:
    grp["age"].plot.kde(ax=axes[1], label=label, color=color,
linewidth=2)
axes[1].axvline(survivors["age"].mean(), color="green",
linestyle="--", linewidth=1)
axes[1].axvline(died["age"].mean(), color="red", linestyle="--",
linewidth=1)
axes[1].set_title("Age Distribution by Survival Outcome")
axes[1].set_xlabel("Age (years)")
axes[1].set_ylabel("Density")
axes[1].legend()

plt.tight_layout()
plt.show()
```

Section 6: Disease Group Analysis

In [ ]:
```
# mortality rate (uses utility) + patient count annotation
grp = df.groupby("dzgroup", observed=True)["death_180d"].agg(["mean", "count
grp.columns = ["Mortality", "N"]
grp = grp.sort_values("Mortality", ascending=False)

fig, ax = plt.subplots(figsize=(10, 5))
plot_mortality_rate(
    df,
    "dzgroup",
    TARGET_COL,
    overall_rate,
    ax,
```

```
    title="180-Day Mortality Rate by Disease Group",
)
# annotate patient counts below percentage labels
for i, (_, row) in enumerate(grp.iterrows()):
    ax.text(
        i,
        row["Mortality"] * 100 - 3.5,
        f"n={row['N']:,}",
        ha="center",
        fontsize=7,
        color="white",
        fontweight="bold",
    )
plt.tight_layout()
plt.show()
```

Section 7: DNR Status

- **no dnr**: no order placed
- **dnr before sadm**: order existed before study admission
- **dnr after sadm**: order placed during the admission (possibly due to detereorating conditions)

In [ ]:
```
DNR_ORDER = ["no dnr", "dnr before sadm", "dnr after sadm"]

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

counts = df["dnr"].value_counts().reindex(DNR_ORDER)

# DNR count
axes[0].bar(range(3), counts.values, edgecolor="white", width=0.6)
axes[0].set_xticks(range(3))
axes[0].set_xticklabels(DNR_ORDER, rotation=15, ha="right", fontsize=8)
axes[0].set_title("Patient Count by DNR Status")
axes[0].set_ylabel("Count")
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 30, f"{v:,}", ha="center", fontsize=9)

# DNR status compare
mort = df.groupby("dnr", observed=True)["death_180d"].mean().reindex(DNR_ORD
hosp = df.groupby("dnr", observed=True)["hospdead"].mean().reindex(DNR_ORDEF
x = range(3)

axes[1].bar(
    [i - 0.2 for i in x],
    mort.values * 100,
    width=0.35,
    color="steelblue",
    edgecolor="white",
    label="180-day mortality",
)
```

```
axes[1].bar(
    [i + 0.2 for i in x],
    hosp.values * 100,
    width=0.35,
    color="lightblue",
    edgecolor="white",
    label="Died in hospital",
)
axes[1].axhline(
    overall_rate * 100,
    color="black",
    linestyle="--",
    linewidth=1.2,
    label=f"Overall mortality ({overall_rate:.1%})",
)
axes[1].set_xticks(list(x))
axes[1].set_xticklabels(DNR_ORDER, rotation=15, ha="right", fontsize=8)
axes[1].set_title("Mortality & Hospital Death Rate by DNR Status")
axes[1].set_ylabel("Rate (%)")
axes[1].legend(fontsize=8)

plt.tight_layout()
plt.show()
```

In [ ]:
```
# DNR rate by Disease Group
dnr_rate = (
    df[df["dnr"] != "no dnr"].groupby("dzgroup", observed=True).size()
    / df.groupby("dzgroup", observed=True).size()
)
dnr_rate = dnr_rate.sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(10, 4))
bars = ax.bar(
    range(len(dnr_rate)),
    dnr_rate.values * 100,
    edgecolor="white",
    width=0.6,
)
ax.set_xticks(range(len(dnr_rate)))
ax.set_xticklabels(dnr_rate.index, rotation=20, ha="right", fontsize=9)
ax.set_ylabel("% of Group with DNR Order")
ax.set_title("DNR Order Rate by Disease Group", fontweight="bold")
plt.show()
```

Section 8: Interaction Analysis — Disease Group × Clinical Features

Per Knaus et al. (1995), lab values and demographic factors only become strongly predictive of mortality in combination with primary disease group. We examine two key interactions identified in the original study:

1. **Albumin × Disease Group** — low albumin was strongly predictive in cancer, COPD, and CHF but relatively unimportant in coma or MOSF
2. **Age × Disease Group** — advancing age heavily impacted COPD survival but had almost no incremental effect in MOSF or malignancy

In [ ]:
```
# Create age groups for interaction analysis
age_bins = [0, 50, 65, 75, 120]
age_labels = ["<50", "50-65", "65-75", "75+"]
df["age_group"] = pd.cut(df["age"], bins=age_bins, labels=age_labels)

# --- Plot: Mortality rate by Disease Group × Age Group ---
age_int = (
    df.groupby(["dzgroup", "age_group"], observed=True)["death_180d"]
    .mean()
    .reset_index()
)
age_int.columns = ["dzgroup", "age_group", "mortality_rate"]

dz_order = (
    df.groupby("dzgroup", observed=True)["death_180d"]
    .mean()
    .sort_values(ascending=False)
    .index.tolist()
)

fig, ax = plt.subplots(figsize=(14, 6))
x = range(len(dz_order))
width = 0.2
age_groups = ["<50", "50-65", "65-75", "75+"]
colors = ["#1a9641", "#a6d96a", "#fdae61", "#d7191c"]

for i, (age_grp, color) in enumerate(zip(age_groups, colors)):
    subset = age_int[age_int["age_group"] == age_grp]
    subset = subset.set_index("dzgroup").reindex(dz_order)
    ax.bar(
        [xi + (i - 1.5) * width for xi in x],
        subset["mortality_rate"] * 100,
        width=width,
        label=age_grp,
        color=color,
        edgecolor="white",
    )

ax.axhline(
    df["death_180d"].mean() * 100,
    color="black",
    linestyle="--",
    linewidth=1.2,
    label=f"Overall ({df['death_180d'].mean():.1%})",
)
ax.set_xticks(list(x))
ax.set_xticklabels(dz_order, rotation=30, ha="right", fontsize=9)
ax.set_ylabel("180-Day Mortality (%)")
ax.set_title(
```

```
    "Mortality Rate by Disease Group × Age Group", fontweight="bold", fontsi
)
ax.legend(title="Age Group", fontsize=8)
plt.tight_layout()
plt.show()
```

8.1 Interaction: Mortality by Disease Group × Age Group

The chart reveals meaningful variation in how age modifies 180-day mortality risk across disease groups, consistent with findings reported by Knaus et al. (1995, Appendix Figure 3).

COPD shows the strongest age gradient. Younger COPD patients (<50, 50–65) fall well below the overall mortality average of 46.8%, while the 65–75 and 75+ groups rise noticeably above it. This confirms the Knaus finding that in COPD, an increase in age from 70 to 75 years meaningfully increases 180-day mortality risk, more so than in any other disease group.

Coma has high baseline mortality regardless of age. Even the youngest Coma patients (~61%) exceed the overall average, indicating that disease severity drives mortality in this group more than age does. A modest amplifying effect of age is visible in the 65–75 to 75+ transition (~77% to ~85%), but this is secondary to the disease itself.

MOSF with Malignancy and Lung Cancer show flat age profiles. Bars are relatively uniform across all age groups, consistent with the Knaus finding that in acute physiologic crises with malignancy, severity of illness dominates prognosis and age adds little incremental risk.

Cirrhosis and CHF show moderate age effects with bars trending upward across age groups but without the steep gradient seen in COPD.

Clinical implication: Age is not uniformly prognostic across disease groups. Its predictive value is disease-dependent, strongest in chronic conditions like COPD where physiologic reserve declines gradually with age, and weakest in acute severe conditions where immediate physiologic status determines outcome. This interaction should be explicitly modeled in the full project rather than treating age as a global linear predictor.

Note: Disease groups are ordered left to right by overall 180-day mortality rate (highest to lowest), not by age effect magnitude.

In [ ]:
```
# Create albumin quartile groups
# KNN imputation restored sufficient variation for 4 true quartiles
# Note: original domain fill of 3.5 coincided with Q3 boundary,
# collapsing bins — replaced with KNN imputation in cleaning notebook
df["alb_quartile"] = pd.qcut(
```

```
    df["alb"], q=4, labels=["Q1 (Low)", "Q2", "Q3", "Q4 (High)"]
)

# --- Plot: Mortality rate by Disease Group × Albumin Quartile ---
alb_int = (
    df.groupby(["dzgroup", "alb_quartile"], observed=True)["death_180d"]
    .mean()
    .reset_index()
)
alb_int.columns = ["dzgroup", "alb_quartile", "mortality_rate"]

fig, ax = plt.subplots(figsize=(14, 6))
quartiles = ["Q1 (Low)", "Q2", "Q3", "Q4 (High)"]
colors = ["#d73027", "#fc8d59", "#91bfdb", "#4575b4"]

for i, (quartile, color) in enumerate(zip(quartiles, colors)):
    subset = alb_int[alb_int["alb_quartile"] == quartile]
    subset = subset.set_index("dzgroup").reindex(dz_order)
    ax.bar(
        [xi + (i - 1.5) * width for xi in x],
        subset["mortality_rate"] * 100,
        width=width,
        label=quartile,
        color=color,
        edgecolor="white",
    )

ax.axhline(
    df["death_180d"].mean() * 100,
    color="black",
    linestyle="--",
    linewidth=1.2,
    label=f"Overall ({df['death_180d'].mean():.1%})",
)
ax.set_xticks(list(x))
ax.set_xticklabels(dz_order, rotation=30, ha="right", fontsize=9)
ax.set_ylabel("180-Day Mortality (%)")
ax.set_title(
    "Mortality Rate by Disease Group × Albumin Quartile", fontweight="bold",
)
ax.legend(title="Albumin Quartile", fontsize=8)
plt.tight_layout()
plt.show()
```

8.2 Interaction: Mortality by Disease Group × Albumin Quartile

Following KNN imputation of albumin (replacing the original domain fill of 3.5 g/dL which collapsed quartile bins), this chart examines whether serum albumin level modifies 180-day mortality risk differently across disease groups, a key interaction identified by Knaus et al. (1995).

Lung Cancer shows the strongest albumin gradient. Mortality decreases from approximately 69% in the lowest albumin quartile (Q1) to 51% in the highest (Q4), an 18 percentage point spread, the largest gradient in the chart. This directly confirms the Knaus finding that low serum albumin had an especially strong relation with risk for death in lung cancer patients.

Colon Cancer shows the second strongest gradient. Mortality decreases from approximately 53% (Q1) to 34% (Q4), a 19 percentage point spread when the full range is considered, though the Q1–Q2 step is the most pronounced. Consistent with Knaus identifying albumin as strongly prognostic in colon cancer patients.

CHF and COPD show weak but present albumin gradients, a modest reduction in mortality from low to high albumin quartiles, consistent with the Knaus finding of a weaker but present albumin association in chronic conditions.

ARF/MOSF with Sepsis and Cirrhosis show relatively flat albumin profiles, suggesting albumin adds limited prognostic value in these groups beyond overall disease severity.

Coma shows an unexpected reverse pattern — Q4 (high albumin) has the highest mortality (~81%) rather than the lowest. This counterintuitive finding likely reflects confounding: patients in a coma with preserved albumin levels may represent a distinct clinical subtype, such as acute neurological events in otherwise well-nourished patients. This warrants further investigation in the modeling phase.

MOSF with Malignancy shows a modest reverse pattern — Q2 is slightly higher than Q1 before declining, suggesting albumin's relationship with mortality is nonlinear or confounded by disease-specific factors in this group.

Clinical and modeling implication: Albumin is not a uniformly useful predictor across all disease groups. Its prognostic value is most pronounced in cancer patients, particularly lung and colon cancer, and largely absent or reversed in acute physiologic crises. The full project model should include albumin × disease group interaction terms rather than treating albumin as a global linear predictor, consistent with the original SUPPORT prognostic model (Knaus et al., 1995).

Methodological note: Albumin was re-imputed using KNN (k=5) with physiologically related features as neighbors (age, meanbp, hrt, resp, temp, sod), replacing the original domain fill value of 3.5 g/dL. The KNN approach produced a clinically coherent distribution (median = 2.96 g/dL, IQR = 2.50–3.40), consistent with expected albumin levels in a critically ill population.

Section 9: Benchmark Evaluation — SUPPORT Model vs. Actual 180-Day Mortality

The SUPPORT prognostic model (Knaus et al., 1995) generated individualized 6-month survival probability estimates (`surv6m`) for each patient at study day 3. A central objective of this project is to compare our classification model's performance against this benchmark.

Before building our model, we first establish how well the SUPPORT estimates discriminate between patients who died within 180 days and those who survived. Three evaluations are performed:

1. **Distribution** — do the SUPPORT survival estimates separate the two outcome groups?
2. **ROC curve and AUC** — how well does the SUPPORT model rank patients by mortality risk? Knaus et al. reported an AUC of 0.78 on phase II validation data; we evaluate whether this holds in our dataset.
3. **Calibration** — do the predicted survival probabilities match actual survival rates across the full probability range?

In [ ]:
```
# Section 9: Benchmark Comparison — SUPPORT Model vs. Actual 180-Day Mortali
#
# The SUPPORT model produced physician-facing survival probability estimates
# (surv6m) for each patient. We evaluate how well these estimates discrimina
# between patients who died within 180 days and those who survived, using:
#
# 1. Distribution plot — surv6m by actual outcome
# 2. ROC curve — with AUC for direct comparison to Knaus et al. (1995)
#    reported AUC of 0.78
# 3. Calibration check — mean predicted survival vs. actual survival rate

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# --- Plot 1: surv6m distribution by actual outcome ---
for label, grp, color in [
    ("Survived", df[df["death_180d"] == 0], "seagreen"),
    ("Died", df[df["death_180d"] == 1], "tomato"),
]:
    grp["surv6m"].plot.kde(ax=axes[0], label=label, color=color, linewidth=2

axes[0].axvline(
    df[df["death_180d"] == 0]["surv6m"].mean(),
    color="seagreen",
    linestyle="--",
    linewidth=1,
    label="Mean (Survived)",
)
axes[0].axvline(
    df[df["death_180d"] == 1]["surv6m"].mean(),
    color="tomato",
```

```
    linestyle="--",
    linewidth=1,
    label="Mean (Died)",
)
axes[0].set_title("SUPPORT Survival Estimate (surv6m)\nby Actual 180-Day Out
axes[0].set_xlabel("Predicted 6-Month Survival Probability")
axes[0].set_ylabel("Density")
axes[0].legend(fontsize=8)

# --- Plot 2: ROC curve ---
# Note: surv6m is probability of SURVIVAL so we invert for death prediction
fpr, tpr, _ = roc_curve(df["death_180d"], 1 - df["surv6m"])
auc = roc_auc_score(df["death_180d"], 1 - df["surv6m"])

axes[1].plot(
    fpr, tpr, color="steelblue", linewidth=2, label=f"SUPPORT model (AUC = {
)
axes[1].plot(
    [0, 1],
    [0, 1],
    color="gray",
    linestyle="--",
    linewidth=1,
    label="Random classifier (AUC = 0.50)",
)
axes[1].axhline(
    0.78,
    color="orange",
    linestyle=":",
    linewidth=1,
    label="Knaus et al. reported AUC = 0.78",
)
axes[1].set_title("ROC Curve — SUPPORT Model\nvs. Actual 180-Day Mortality")
axes[1].set_xlabel("False Positive Rate")
axes[1].set_ylabel("True Positive Rate")

fpr2m, tpr2m, _ = roc_curve(df["death_180d"], 1 - df["surv2m"])
auc2m = roc_auc_score(df["death_180d"], 1 - df["surv2m"])
axes[1].plot(
    fpr2m,
    tpr2m,
    color="tomato",
    linewidth=1.5,
    linestyle="--",
    label=f"surv2m — 2-month estimate (AUC = {auc2m:.3f})",
)

# Legend called last so all lines are included
axes[1].legend(fontsize=8)

# --- Plot 3: Calibration ---
# Bin patients by predicted survival probability and compare to actual survi
df["surv6m_bin"] = pd.cut(df["surv6m"], bins=10)
calibration = (
    df.groupby("surv6m_bin", observed=True)
    .agg(
```

```
        mean_predicted=("surv6m", "mean"),
        actual_survival=("death_180d", lambda x: 1 - x.mean()),
    )
    .dropna()
)

axes[2].scatter(
    calibration["mean_predicted"],
    calibration["actual_survival"],
    color="steelblue",
    s=80,
    zorder=3,
)
axes[2].plot(
    [0, 1],
    [0, 1],
    color="gray",
    linestyle="--",
    linewidth=1,
    label="Perfect calibration",
)
axes[2].set_title(
    "Reliability Diagram — SUPPORT Model\nPredicted vs. Actual Survival Rate
)
axes[2].set_xlabel("Mean Predicted Survival (surv6m)")
axes[2].set_ylabel("Actual Survival Rate")
axes[2].legend(fontsize=8)

plt.suptitle(
    "Section 9: SUPPORT Model Benchmark Evaluation",
    fontsize=13,
    fontweight="bold",
    y=1.02,
)
plt.tight_layout()
plt.show()

# Summary stats
print(f"SUPPORT model AUC: {auc:.3f}")
print("Knaus et al. reported AUC: 0.780")
print(f"\nMean surv6m — Survived: {df[df['death_180d'] == 0]['surv6m'].mean(
print(f"Mean surv6m — Died:     {df[df['death_180d'] == 1]['surv6m'].mean():
print(f"\nOverall actual survival rate: {1 - df['death_180d'].mean():.3f}")
print(f"Overall mean surv6m:          {df['surv6m'].mean():.3f}")
print(f"\nsurv6m AUC: {auc:.3f}")
print(f"surv2m AUC: {auc2m:.3f}")
```

9.1 Benchmark Evaluation Results

Distribution of SUPPORT Predictions by Actual Outcome

The SUPPORT model assigned meaningfully different survival probabilities to patients who survived versus those who died. Survivors received a mean predicted survival probability of 0.640, while patients who died received a mean

of 0.389 — a separation of 0.251. The distributions overlap substantially in the 0.3–0.7 range, reflecting the inherent uncertainty in predicting outcomes for patients with moderate risk profiles. The x-axis extends slightly beyond the valid 0–1 probability range at both extremes due to kernel density smoothing, not real data values.

Discrimination — ROC Curve and AUC

The ROC curve plots two independently calculated metrics across all possible decision thresholds:

- **True Positive Rate (y-axis):** of all patients who actually died, what fraction did the model correctly identify as high risk?
- **False Positive Rate (x-axis):** of all patients who actually survived, what fraction did the model incorrectly flag as high risk?

Each point on the curve represents one threshold setting, showing the simultaneous tradeoff between catching actual deaths and generating false alarms. The two metrics have no direct mathematical relationship; they are calculated from entirely separate patient groups and plotted together to visualize the tradeoff as the threshold moves from 0 to 1.

The area under the curve (AUC) summarizes this tradeoff in a single number: the probability that the model ranks a randomly chosen patient who died as higher risk than a randomly chosen patient who survived. An AUC of 0.787 means the SUPPORT model makes this correct ranking approximately 79% of the time, replicating Knaus et al.'s reported AUC of 0.78 on independent phase II validation data with remarkable fidelity across 30 years and a different sample. This consistency also serves as indirect validation that our target variable construction and cleaning approach are methodologically sound.

AUC is a threshold-agnostic metric; it does not indicate which specific threshold to use clinically, nor does it reflect performance at any single operating point. For clinical decision making, where the cost of missing a death typically exceeds the cost of a false alarm, precision-recall analysis at a specific threshold is more informative. This will be addressed in the modeling notebook.

Reliability — Predicted vs. Actual Survival

The reliability diagram assesses whether the SUPPORT model's probability estimates are trustworthy in absolute terms; when the model assigns a 70% survival probability, do approximately 70% of those patients actually survive? This is distinct from discrimination: a model can rank patients correctly (high AUC) while producing poorly calibrated probabilities, or vice versa.

The dots closely follow the perfect calibration diagonal across the full probability range, indicating the SUPPORT model's survival estimates are reliable in absolute terms. The overall mean predicted survival (0.523) closely matches the actual survival rate (0.534), a gap of only 1.1 percentage points, confirming good global calibration.

Note that this is a calibration *assessment*, not a recalibration step. No correction to the model's probabilities has been applied. Formal calibration metrics (Brier score, Hosmer-Lemeshow test) will be computed for our own model in the modeling notebook.

Benchmark Summary

The SUPPORT model demonstrates both strong discrimination (AUC = 0.787) and reliable probability estimates across the full risk range. This establishes a meaningful performance benchmark: our classification model must approach or exceed AUC = 0.787 to be considered competitive with the original SUPPORT prognostic model. Combined discrimination and calibration performance will be the primary basis for that comparison.

surv2m vs. surv6m Discrimination

The 2-month survival estimate (`surv2m`) was evaluated alongside `surv6m` as an additional benchmark. Despite measuring a different time horizon, `surv2m` produced a nearly identical AUC of 0.781 — a difference of 0.006 from `surv6m` . The two ROC curves are visually indistinguishable. This confirms the high correlation between the two estimates (r = 0.960) observed in the correlation matrix and formally justifies the use of `surv6m` as the primary benchmark — not because `surv2m` performs meaningfully worse, but because `surv6m` directly aligns with our 180-day target window.

Section 10: Categorical Features: Chi-Squared Tests & Contingency Tables

Tests whether a categorical variable is **statistically independent** of 180-day mortality.

$$H_0 : \text{the variable is independent of death\_180d}$$

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Reject H_0 when $p < 0.05$.

Standardized residuals = $(O - E)/\sqrt{E}$ — cells with $|r| > 2$ deviate significantly from independence and drive the chi-squared result.

10.1 Race and Sex shows low P significance

In [ ]:
```
import numpy as np
from scipy.stats import chi2_contingency

CAT_COLS = ["sex", "dzgroup", "dzclass", "race", "ca", "dnr"]

rows = []
for col in CAT_COLS:
    ct = pd.crosstab(df[col], df[TARGET_COL])
    chi2, p, dof, _ = chi2_contingency(ct)
    rows.append(
        {
            "Variable": col,
            "Chi2 Stat": round(chi2, 2),
            "df": dof,
            "p-value": f"{p:.2e}",
            "Significant": "Yes" if p < 0.05 else "No",
        }
    )

chi2_df = pd.DataFrame(rows).sort_values("Chi2 Stat", ascending=False)
print("Chi-Squared Test Summary:")
print(chi2_df.to_string(index=False))
```

10.2 Within Disease Group and Cancer, which class affects mortality most?

- Disease Group: MOSF w/ malignant, Coma, Lung Cancer (in that order) die more than overall rate
- Cancer: metatastic patients die more than the overall rate

In [ ]:
```
# Standardized residuals heatmaps
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

for ax, col in zip(axes, ["dzgroup", "ca"]):
    ct = pd.crosstab(df[col], df[TARGET_COL])
    ct.columns = ["Survived", "Died"]
    _, _, _, exp = chi2_contingency(ct)
    std_resid = (ct.values - exp) / np.sqrt(exp)
    std_resid_df = pd.DataFrame(std_resid, index=ct.index, columns=ct.columr

    im = ax.imshow(std_resid_df.values, cmap="RdBu_r", vmin=-6, vmax=6, aspe
    ax.set_xticks(range(len(ct.columns)))
    ax.set_xticklabels(ct.columns)
    ax.set_yticks(range(len(ct.index)))
    ax.set_yticklabels(ct.index, fontsize=9)
    ax.set_title(
        f"Standardized Residuals: {col}\n|r| > 2 drives chi-squared",
```

```
        fontweight="bold",
    )

    for i in range(std_resid_df.shape[0]):
        for j in range(std_resid_df.shape[1]):
            v = std_resid_df.iloc[i, j]
            ax.text(
                j,
                i,
                f"{v:.1f}",
                ha="center",
                va="center",
                fontsize=9,
                color="black" if abs(v) < 3 else "white",
                fontweight="bold",
            )

    plt.colorbar(im, ax=ax, label="Std. Residual")

fig.suptitle(
    "Standardized Residuals: Cells Driving Chi-Squared",
    fontsize=12,
    fontweight="bold",
)
plt.tight_layout()
plt.show()
```

In [ ]:

```
decisions = pd.DataFrame(
    [
        {
            "Feature": "sex",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "~1-2% mortality gap between male/female; barely ab
        },
        {
            "Feature": "edu",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "Weak gradient; 18% of values imputed (median=12 ir
        },
        {
            "Feature": "hday",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "Day entered study; no clear causal link to 180-day
        },
        {
            "Feature": "charges",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Hospital charges — administrative, not a clinical
        },
        {
            "Feature": "totcst",
            "Decision": "Drop",
```

```
            "Signal": "None",
            "Rationale": "Total RCC cost — administrative, not a clinical pr
        },
        {
            "Feature": "totmcst",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Total micro-cost — administrative, 38% imputed",
        },
        {
            "Feature": "All day-3 labs",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Primary clinical predictors per Knaus et al. (1995
        },
        {
            "Feature": "age, adlsc, scoma",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Strong EDA effect sizes and clear clinical mechani
        },
        {
            "Feature": "dzgroup / dzclass",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Largest mortality spread of any categorical featur
        },
        {
            "Feature": "ca, diabetes, dementia",
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Comorbidities with known mortality associations",
        },
        {
            "Feature": "income, race",
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Socioeconomic signal; moderate mortality gradient
        },
    ]
)

print("Feature Selection Decisions:")
print(decisions.to_string(index=False))
```

In [ ]:

```
decisions = pd.DataFrame(
    [
        {
            "Feature": "sex",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "~1-2% mortality gap between male/female; barely ab
        },
        {
            "Feature": "edu",
            "Decision": "Drop",
```

```
            "Signal": "Weak",
            "Rationale": "Weak gradient; 18% of values imputed (median=12 ir
        },
        {
            "Feature": "hday",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "Day entered study; no clear causal link to 180-day
        },
        {
            "Feature": "charges",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Hospital charges — administrative, not a clinical
        },
        {
            "Feature": "totcst",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Total RCC cost — administrative, not a clinical pr
        },
        {
            "Feature": "totmcst",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Total micro-cost — administrative, 38% imputed",
        },
        {
            "Feature": "All day-3 labs",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Primary clinical predictors per Knaus et al. (1995
        },
        {
            "Feature": "age, adlsc, scoma",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Strong EDA effect sizes and clear clinical mechani
        },
        {
            "Feature": "dzgroup / dzclass",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Largest mortality spread of any categorical featur
        },
        {
            "Feature": "ca, diabetes, dementia",
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Comorbidities with known mortality associations",
        },
        {
            "Feature": "income, race",
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Socioeconomic signal; moderate mortality gradient
```

```
        },
    ]
)

print("Feature Selection Decisions:")
print(decisions.to_string(index=False))
```

Section 9: Additional Feature Pruning after EDA

Based on our EDA we are dropping `edu`. We are keeping `sex` despite its weak standalone signal, because it stays in as a control (a demographic confounder) for the inference models, so the disease-group odds ratios are adjusted for it.

Column(s)	Decision	Rationale
`edu`	**Drop**	Weak significance based on Bivariate and Chi-Squared analysis
`sex`	**Keep as control**	Weak standalone signal, kept as a demographic confounder for the inference models

In [ ]:
```
DROP_COLS = ["edu"]

MODEL_FEATURE_COLS = [c for c in FEATURE_COLS if c not in DROP_COLS]

print(f"Features available in FEATURE_COLS : {len(FEATURE_COLS)}")
print(f"Features dropped                   : {len(DROP_COLS)}  {DROP_COLS}")
print(f"Final model feature count          : {len(MODEL_FEATURE_COLS)}")
print()
print("Final model features:")
for col in MODEL_FEATURE_COLS:
    print(f"  {col}")
```

In [ ]:
```
OUTPUT_MODEL = _proj_root / "data" / "support2_model_features.csv"
df[MODEL_FEATURE_COLS + [TARGET_COL]].to_csv(OUTPUT_MODEL, index=False)

n_feat = len(MODEL_FEATURE_COLS)
print(f"Saved : {OUTPUT_MODEL}")
print(f"Shape : {df.shape[0]:,} rows  x  {n_feat + 1} columns")
print(f"  Features : {n_feat}  |  Target : {TARGET_COL}")
print()
print("Data layers:")
print("  Bronze : support2_raw_complete.csv  (raw download)")
print("  Silver : support2_cleaned.csv       (imputed, all columns)")
print("  Gold   : support2_model_features.csv (feature-selected, model-ready
```

Section 12: Correlation Matrix — Numeric Features

With 46 variables available for modeling, multicollinearity is a meaningful risk. Highly correlated features provide redundant information and can destabilize model coefficients, inflate standard errors, and complicate interpretation. This section identifies problematic feature pairs before modeling begins.

The correlation matrix displays Pearson correlations between all numeric features. Values close to 1.0 indicate strong positive correlation, values close to -1.0 indicate strong negative correlation, and values near 0 indicate little linear relationship. Pairs with |r| > 0.7 are flagged as candidates for feature selection or dimensionality reduction in the modeling notebook.

In [ ]:
```
# Section 12: Correlation Heatmap — Numeric Features
#
# With 40+ potential predictors, multicollinearity is a real risk.
# Highly correlated features (|r| > 0.7) provide redundant information
# and can destabilize model coefficients. This heatmap identifies
# problematic pairs before modeling begins.

numeric_cols = df.select_dtypes(include="number").columns.tolist()

# Exclude target, outcome, and identifier columns
exclude = ["death_180d", "death", "hospdead", "id", "slos", "d_time"]
feature_cols = [c for c in numeric_cols if c not in exclude]

corr_matrix = df[feature_cols].corr()

# Mask upper triangle for cleaner reading

mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

fig, ax = plt.subplots(figsize=(18, 14))
sns.heatmap(
    corr_matrix,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    vmin=-1,
    vmax=1,
    linewidths=0.5,
    annot_kws={"size": 7},
    ax=ax,
)
ax.set_title(
    "Correlation Matrix — Numeric Features", fontsize=14, fontweight="bold",
)
plt.xticks(rotation=45, ha="right", fontsize=8)
plt.yticks(fontsize=8)
plt.tight_layout()
plt.show()

# Flag high correlation pairs
```

```
print("=== High Correlation Pairs (|r| > 0.7) ===\n")
high_corr = []
for i in range(len(corr_matrix.columns)):
    for j in range(i):
        r = corr_matrix.iloc[i, j]
        if abs(r) > 0.7:
            high_corr.append(
                {
                    "Feature A": corr_matrix.columns[i],
                    "Feature B": corr_matrix.columns[j],
                    "r": round(r, 3),
                }
            )

if high_corr:
    high_corr_df = pd.DataFrame(high_corr).sort_values("r", ascending=False)
    print(high_corr_df.to_string(index=False))
else:
    print("No pairs with |r| > 0.7 found.")
```

12.1 Correlation Findings

The majority of numeric features are weakly correlated with one another (|r| < 0.5), indicating the feature set is generally clean and suitable for modeling. Five pairs exceed the |r| > 0.7 multicollinearity threshold and warrant specific attention.

`surv6m` × `surv2m` (r = 0.960) — the strongest correlation in the matrix. Both are SUPPORT model survival probability estimates differing only in time horizon (6-month vs. 2-month). They are measuring the same underlying construct from the same model inputs, making them functionally redundant as predictors. `surv2m` should be excluded from the feature set; `surv6m` is retained as the benchmark comparison variable and excluded from model features to avoid leakage.

`prg6m` × `prg2m` (r = 0.889) — physician survival estimates at 6 and 2 months respectively. The same redundancy applies as with the SUPPORT estimates; both capture physician judgment about the same patient at the same point in time. `prg2m` should be excluded; `prg6m` retained if physician estimates are included as a feature, noting that combining physician estimates with model features replicates the physician-enhanced SUPPORT model described in Knaus et al. (1995).

`aps` × `sps` (r = 0.800) — both are composite physiologic severity scores constructed from overlapping input variables including blood pressure, heart rate, respiratory rate, temperature, and laboratory values. Their correlation reflects shared measurement of the same underlying construct (severity of acute illness) rather than any methodological dependency. Unlike the survival estimate

pairs, `aps` and `sps` apply different disease-specific weightings and are not perfectly interchangeable. Both are retained as candidates for the feature set; LASSO regularization in the modeling phase will naturally downweight the less informative of the two.

`totcst` × `charges` (r = 0.742) — total cost and hospital charges are financially related by definition. More importantly, both are outcomes of care delivery rather than clinical predictors of mortality; a patient does not incur charges because they are sick...they incur charges because of interventions performed. Including either as a model feature would introduce reverse causality. Both should be excluded from the feature set entirely.

`surv2m` × `sps` (r = -0.756) — a strong negative correlation between the 2-month survival estimate and the physiology severity score. This is clinically expected and not a multicollinearity concern; sicker patients (higher sps) receive lower survival probability estimates. The negative sign confirms the directionality is correct. No action required.

Modeling implications: Three actions follow from this analysis:

1. Drop `surv2m` and `prg2m` — redundant with their 6-month counterparts
2. Drop `totcst` and `charges` — outcomes of care, not predictors
3. Monitor `aps` and `sps` — retain both but apply LASSO regularization to manage their shared variance in the modeling notebook

---

Section 13: Numeric Feature Summary by Outcome Group

The following table presents mean values of key clinical variables stratified by 180-day mortality outcome. Features are ordered by the standardized difference between groups, which normalizes raw differences by pooled standard deviation, putting all variables on the same scale regardless of units.

This table serves two purposes: it provides a reportable summary of group differences for the technical report, and it establishes the basis for formal hypothesis testing in Section 13.1, where statistical significance of each difference is evaluated.

In [ ]:
```
# Section 13: Numeric Summary by Outcome Group
#
# Mean values of key clinical variables broken out by 180-day mortality outc
# Standardized difference (Cohen's d approximation) normalizes raw differenc
# by pooled standard deviation, putting all variables on the same scale
# regardless of units. This corrects for the misleading ranking produced by
```

```python
# sorting on raw differences alone.

key_features = [
    "age",
    "num_co",
    "scoma",
    "sps",
    "aps",
    "meanbp",
    "hrt",
    "resp",
    "temp",
    "alb",
    "bili",
    "crea",
    "sod",
    "ph",
    "glucose",
    "bun",
    "wblc",
    "pafi",
    "urine",
    "adlsc",
]

summary_std = df.groupby("death_180d")[key_features].agg(["mean", "std"])

rows = []
for feat in key_features:
    mean_surv = summary_std.loc[0, (feat, "mean")]
    mean_died = summary_std.loc[1, (feat, "mean")]
    std_surv = summary_std.loc[0, (feat, "std")]
    std_died = summary_std.loc[1, (feat, "std")]

    pooled_std = np.sqrt((std_surv**2 + std_died**2) / 2)

    raw_diff = mean_died - mean_surv
    std_diff = raw_diff / pooled_std if pooled_std > 0 else 0

    rows.append(
        {
            "Feature": feat,
            "Survived": round(mean_surv, 3),
            "Died": round(mean_died, 3),
            "Raw Difference": round(raw_diff, 3),
            "Standardized Difference": round(std_diff, 3),
        }
    )

summary_final = pd.DataFrame(rows).set_index("Feature")
summary_final = summary_final.reindex(
    summary_final["Standardized Difference"].abs().sort_values(ascending=Fal
)

print("=== Numeric Feature Means by 180-Day Mortality Outcome ===\n")
print(summary_final.to_string())
```

Section 13.1: Hypothesis Testing — Group Differences by Outcome

The summary table in Section 13 identified univariate differences in feature means between outcome groups. This section formally tests whether those differences are statistically significant or could plausibly have occurred by chance.

Note on inference in EDA: Hypothesis testing is inferential rather than exploratory. It is included here as a bridge between EDA and modeling. Features with statistically significant differences between outcome groups are stronger candidates for the feature set. Findings should be interpreted as hypothesis-generating rather than confirmatory; formal model-based inference follows in the modeling notebook.

Test selection: Most clinical variables in this dataset are right-skewed, as confirmed in earlier EDA. The Mann-Whitney U test is used rather than the independent samples t-test because it makes no assumption of normality and is more appropriate for skewed continuous data. The null hypothesis for each feature is that the distribution is identical between survivors and patients who died. A Bonferroni correction is applied to account for multiple comparisons across 20 features, yielding an adjusted significance threshold of $\alpha = 0.05 / 20 = 0.0025$.

In [ ]:
```
# Section 13.1: Mann-Whitney U Tests — Feature Differences by Outcome
#
# Tests whether each feature's distribution differs significantly between
# survivors and patients who died. Mann-Whitney U is used over t-test
# because most clinical variables are right-skewed.
# Bonferroni correction applied for multiple comparisons (n=20 features).

survived = df[df["death_180d"] == 0]
died = df[df["death_180d"] == 1]

alpha = 0.05
bonferroni_alpha = alpha / len(key_features)

results = []
for feat in key_features:
    stat, p = stats.mannwhitneyu(
        survived[feat].dropna(), died[feat].dropna(), alternative="two-sidec
    )
    results.append(
        {
            "Feature": feat,
            "Survived Mean": round(survived[feat].mean(), 3),
            "Died Mean": round(died[feat].mean(), 3),
            "Standardized Diff": round(
                summary_final.loc[feat, "Standardized Difference"], 3
```

```
            ),
            "U Statistic": round(stat, 1),
            "p-value": round(p, 6),
            "Significant (Bonferroni)": "Yes" if p < bonferroni_alpha else "
        }
    )

results_df = pd.DataFrame(results).set_index("Feature")
results_df = results_df.reindex(summary_final.index)

print(
    f"Mann-Whitney U Test Results (Bonferroni-corrected α = {bonferroni_alph
)
print(results_df.to_string())

sig = (results_df["Significant (Bonferroni)"] == "Yes").sum()
not_sig = (results_df["Significant (Bonferroni)"] == "No").sum()
print(f"\nSignificant features: {sig} / {len(key_features)}")
print(f"Non-significant features: {not_sig} / {len(key_features)}")
```

13.2 Hypothesis Testing Results

Thirteen of 20 features showed statistically significant differences between survivors and patients who died after Bonferroni correction (adjusted α = 0.0025). Seven features failed to reach significance and are unlikely to contribute meaningful univariate predictive value.

Significant features (13): sps, aps, scoma, adlsc, age, bili, hrt, alb, meanbp, crea, pafi, wblc, resp. All returned p-values at or near zero with the exception of bili (p = 0.000001), crea (p = 0.000002), pafi (p = 0.000001), wblc (p = 0.000204), and resp (p = 0.000490), all well below the corrected threshold.

Non-significant features (7): temp, num_co, bun, urine, ph, sod, glucose. These features show negligible standardized differences and p-values ranging from 0.150 to 0.937, providing no statistical evidence of distributional difference between outcome groups.

Notable findings:

The three strongest separators by standardized difference, sps (0.664), aps (0.597), and scoma (0.522), are also the most statistically significant, with p-values indistinguishable from zero. This convergence of effect size and significance strengthens the case for their inclusion in the modeling feature set and is consistent with Knaus et al. (1995) ranking physiology score and coma score as the two most important prognostic factors.

Albumin (alb) is statistically significant ($p < 0.0001$) despite a modest standardized difference of -0.148. This apparent contradiction is resolved by the interaction analysis in Section 8, where albumin's predictive value is

concentrated in specific disease groups rather than distributed uniformly across the population.

The non-significance of glucose, bun, ph, and sod is noteworthy given their clinical relevance in critical illness. This likely reflects the impact of median imputation on these variables, glucose (49% imputed), bun (48% imputed), and ph (25% imputed), which compresses their distributions toward the median, reducing apparent group differences. These findings should be interpreted cautiously and revisited if imputation strategy changes in the full project.

The non-significance of num_co ($p = 0.429$) is consistent with the Knaus et al. finding that the only statistically significant comorbidity was diagnosis of malignancy, captured here by disease group rather than comorbidity count.

Modeling recommendations: Features with p-values above the Bonferroni threshold (temp, num_co, bun, urine, ph, sod, glucose) are candidates for exclusion from the initial feature set. The following features must be excluded regardless of statistical significance due to data leakage:

- `sps` — direct component of the SUPPORT physiology score; using it to predict the same outcome the SUPPORT model predicts is circular
- `surv6m`, `surv2m` — SUPPORT model output predictions; retained as benchmark comparison variables only, never as model features
- `prg6m`, `prg2m` — physician survival estimates generated from the same clinical assessments captured by other features; including them would conflate clinical judgment with physiologic measurement

`aps` (APACHE III score) is an independent benchmark system and does not constitute leakage in the same way, but its inclusion should be explicitly justified during feature selection given its role as a competing prognostic instrument rather than a raw clinical measurement.

For all remaining significant features, univariate significance is a necessary but not sufficient criterion for inclusion. Features that are non-significant individually may contribute in combination with others. Final feature selection will be informed by regularization in the modeling notebook.

Section 10: Univariable screen

This screen ranks each day-3 feature by its standalone association with 180-day mortality, as a first step toward narrowing the candidate predictors before the multivariable model. The metric is a rank-based AUROC, where 0.50 indicates no separation between survivors and non-survivors.

In [ ]:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score

# univariable screen: rank each day-3 feature by how well it separates
# survivors from non-survivors on its own, before the multivariable model
# reuse df loaded at the top of the notebook (PR #9: no redundant load)
y = df["death_180d"]

# candidate day-3 features (severity scores, vitals, labs)
variables = [
    "aps", "sps", "scoma", "age", "meanbp", "hrt", "resp", "temp",
    "wblc", "crea", "bili", "alb", "pafi", "bun", "sod", "ph", "glucose",
]
variables = [v for v in variables if v in df.columns]


def univariable_auroc(x, y):
    # AUROC = the chance a non-survivor has a higher value than a survivor.
    # 0.50 = no separation. Drop missing rows; max(auc, 1 - auc) keeps a
    # protective feature (low value = worse) showing its strength.
    mask = pd.Series(x).notna()
    auc = roc_auc_score(y[mask], x[mask])
    return max(auc, 1 - auc)


scores = pd.Series(
    {v: univariable_auroc(df[v], y) for v in variables}
).sort_values(ascending=False)
print(scores.round(2).to_string())

# bar chart of each feature's standalone association strength
scores[::-1].plot.barh(figsize=(7, 5), color="#4C72B0")
plt.axvline(0.5, color="gray", linestyle="--", linewidth=1)
plt.xlabel("Univariable association (AUROC, 0.50 = chance)")
plt.title("Univariable screen: day-3 variables vs 180-day mortality")
plt.tight_layout()
plt.show()
```

The severity composites show the strongest univariable associations (sps 0.68, aps 0.66, scoma 0.62), while the individual vitals and laboratory values cluster between 0.50 and 0.54. SPS and APS are excluded from the candidate predictor set on methodological grounds: SPS is derived within the original SUPPORT study (a data-leakage risk), and APS varies by institution (which limits generalizability). Both are retained instead as benchmark candidates for the calibration sub-analysis, which compares the legacy SUPPORT prognostic outputs against observed outcomes. SCOMA remains a valid candidate predictor, and the individual labs will need the multivariable model to show whether they add value beyond the composite scores and the disease group.

SUPPORT2 Dataset: Model Selection (Andre)

AAI-500 Applied Statistics for AI | Final Team Project — Group 3

Evaluating Early Clinical Indicators for 180-Day Mortality: A Comparative Analysis Using the SUPPORT2 Dataset

---

Notebook Objectives

This notebook covers the **Model Selection** phase of the project pipeline:

1. Load model-ready feature set and apply feature exclusions
2. Preprocessing: scale numeric features, encode categoricals
3. Baseline logistic regression (no regularization)
4. LASSO-regularized logistic regression (L1 penalty, tuned)
5. Random Forest classifier (tuned)
6. ROC curve comparison: all models vs. SUPPORT benchmark (AUC = 0.787)
7. Model selection and justification

Benchmark: Knaus et al. (1995) reported an AUC of 0.78 for the SUPPORT prognostic model on independent phase II validation data. Our EDA confirmed this holds in the current dataset (AUC = 0.787 using `surv6m`). Matching or exceeding this AUC is the primary performance target.

Reference: Knaus, W. A., Harrell, F. E., Lynn, J., Goldman, L., Phillips, R. S., Connors, A. F., ... & Wagner, D. P. (1995). The SUPPORT prognostic model: Objective estimates of survival for seriously ill hospitalized adults. *Annals of Internal Medicine, 122*(3), 191–203.

Section 0: Imports

In [ ]:
```
import sys
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
```

```
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_tes
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardSca
from sklearn.compose import ColumnTransformer

sys.path.append("../../utils")
from dataset import load_csv

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
SUPPORT_AUC = 0.787  # Knaus et al. (1995), confirmed in EDA notebook Sectic
```

Section 1: Load Data & Feature Selection

In [ ]:
```
df = load_csv("support2_model_features.csv")
print(f"Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
df.head()
```

1.1 Feature Exclusions

The following columns are excluded from the feature set on methodological grounds.

Leakage: prognostic system outputs (`sps`, `surv6m`, `surv2m`, `prg6m`, `prg2m`, `aps`)

These columns are outputs of competing prognostic systems, not raw clinical inputs. Including them would would test our models' ability to replicate existing scoring systems rather than predict mortality from clinical data.

- `sps`: SUPPORT physiology sub-score, a direct component of the SUPPORT model whose output (`surv6m`) is our benchmark. Including it would be circular.
- `surv6m`, `surv2m`: SUPPORT model survival probability outputs. Retained in `support2_cleaned.csv` for benchmark comparison; excluded here because they ARE the benchmark.
- `prg6m`, `prg2m`: physician survival estimates collected at study day 3. Including them replicates the physician-enhanced SUPPORT model rather than an independent classifier.

- `aps`: APACHE III composite severity score. Knaus et al. (1995) evaluated SUPPORT and APACHE III as parallel competing systems (Table 2); neither used the other's score as an input feature. Including `aps` has no methodological precedent in the source paper.
- `avtisst`: average TISS score days 3-25. Spans the full observation period rather than capturing day-3 status alone. The average is partially determined by post-day-3 clinical trajectory, which is downstream of the outcome.
- `sfdm2`: 2-month functional disability score assessed via patient and surrogate questionnaires after the observation period. Classified as a Target in the data dictionary. Measuring outcome status at 2 months makes it downstream of the prediction window for patients who died before that point.

Administrative cost variables (`charges`, `totcst`, `totmcst`)

Excluded on two grounds:

1. **Temporal leakage**: costs accumulate during the hospitalization as a consequence of care delivered. They reflect illness trajectory and intervention intensity rather than predicting mortality independently. The variable is downstream of the outcome, not upstream of it.
2. **Redundancy**: `totcst` and `charges` are highly correlated (r = 0.742, EDA Section 12), carrying no independent signal from one another.

Non-clinical enrollment variable (`hday`)

`hday` records the day the patient entered the study. It reflects enrollment timing, not clinical status at day 3, and has no causal pathway to 180-day mortality.

In [ ]:
```
TARGET = "death_180d"  # exclude the target variable itself
LEAKAGE_COLS = [
    "sps",
    "surv6m",
    "surv2m",
    "prg6m",
    "prg2m",
    "aps",
    "avtisst",
    "sfdm2",
]
ADMIN_COLS = ["charges", "totcst", "totmcst"]
NON_CLINICAL_COLS = ["hday"]

feature_cols = [
    c
    for c in df.columns
    if c != TARGET
    and c not in LEAKAGE_COLS
    and c not in ADMIN_COLS
```

```
    and c not in NON_CLINICAL_COLS
]

X = df[feature_cols]
y = df[TARGET]

print(f"Features: {len(feature_cols)}")
print(f"Target: {TARGET}")
print("\nClass balance:")
print(
    y.value_counts(normalize=True)
    .rename({0: "Survived", 1: "Died"})
    .rename_axis(None)
    .to_string()
)
```

1.2 Train/Test Split

In [ ]:
```
# Stratified split preserves class balance across train and test sets.
# 80/20 split gives sufficient training data while holding out a meaningful
# evaluation set. random_state ensures reproducibility across runs.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE
)

print(f"Train: {X_train.shape[0]:,} rows")
print(f"Test: {X_test.shape[0]:,} rows")
print(f"\nTrain class balance: {y_train.mean():.4f} positive rate")
print(f"Test class balance: {y_test.mean():.4f} positive rate")
```

Section 2: Preprocessing

In [ ]:
```
# Identify column types for preprocessing routing.
# Nominal categoricals get one-hot encoding (no ordinal assumption).
# Ordinal categoricals get ordinal encoding (preserves rank order from clear
# All numeric features get standard scaling (required for logistic regressio

NOMINAL_CATS = ["sex", "dzgroup", "dzclass", "race", "ca", "dnr"]
ORDINAL_CATS = ["income"]  # sfdm2 removed (classified as Target in data dic

# Ordinal category orders established in [EDA REF]
INCOME_ORDER = ["under $11k", "$11-$25k", "$25-$50k", ">$50k"]

# Filter to only columns present in X after leakage exclusion
nominal_present = [c for c in NOMINAL_CATS if c in feature_cols]
ordinal_present = [c for c in ORDINAL_CATS if c in feature_cols]
numeric_present = [c for c in feature_cols if c not in NOMINAL_CATS + ORDINA

print(f"Numeric features: {len(numeric_present)}  {numeric_present}")
print(f"Nominal features: {len(nominal_present)}  {nominal_present}")
print(f"Ordinal features: {len(ordinal_present)}  {ordinal_present}")
```

In [ ]:
```
# ColumnTransformer applies different preprocessing to each column type in c
# drop='first' on OneHotEncoder avoids the dummy variable trap in logistic r
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_present),
        (
            "nom",
            OneHotEncoder(drop="first", handle_unknown="ignore", sparse_outp
            nominal_present,
        ),
        (
            "ord",
            OrdinalEncoder(
                categories=[INCOME_ORDER],
                handle_unknown="use_encoded_value",
                unknown_value=-1,
            ),
            ordinal_present,
        ),
    ],
    remainder="drop",
)

print("Preprocessor built successfully.")
```

Section 3: Baseline Logistic Regression

Logistic regression with no regularization establishes a performance floor. It is interpretable, fast, and directly comparable to the Cox proportional hazards model used by Knaus et al. (1995), which shares the same log-odds foundation. A weak baseline result motivates regularization or ensemble methods; a strong baseline suggests the outcome is largely linearly separable in feature space.

In [ ]:
```
pipe_lr = Pipeline(
    [
        ("preprocessor", preprocessor),
        (
            "classifier",
            LogisticRegression(
                penalty=None,  # no regularization — true baseline
                solver="lbfgs",
                max_iter=1000,
                random_state=RANDOM_STATE,
            ),
        ),
    ]
)

pipe_lr.fit(X_train, y_train)
```

```
y_pred_lr = pipe_lr.predict(X_test)
y_prob_lr = pipe_lr.predict_proba(X_test)[:, 1]

metrics_lr = {
    "Model": "Logistic Regression (baseline)",
    "Accuracy": accuracy_score(y_test, y_pred_lr),
    "Precision": precision_score(y_test, y_pred_lr),
    "Recall": recall_score(y_test, y_pred_lr),
    "F1": f1_score(y_test, y_pred_lr),
    "ROC AUC": roc_auc_score(y_test, y_prob_lr),
}

print("Baseline Logistic Regression: Test Set Performance")
for k, v in metrics_lr.items():
    if k != "Model":
        print(f"  {k:<12} {v:.4f}")
print(f"\n  SUPPORT benchmark AUC: {SUPPORT_AUC:.3f}")
```

In [ ]:
```
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_lr),
    display_labels=["Survived", "Died"],
).plot(ax=ax, colorbar=False)
ax.set_title("Confusion Matrix: Baseline Logistic Regression", fontweight="b
plt.tight_layout()
plt.show()
```

Section 4: LASSO Logistic Regression

L1 (LASSO) regularization shrinks less informative coefficients to exactly zero, performing implicit feature selection. This is appropriate here because:

1. Seven features were non-significant in EDA hypothesis testing (temp, num_co, bun, urine, ph, sod, glucose); LASSO can downweight these without manual exclusion
2. Correlated feature pairs (aps × sps, already excluded) can cause coefficient instability in unregularized logistic regression
3. The resulting sparse model is more interpretable for a clinical audience

Note: reference categories for nominal features are determined by alphabetical ordering via scikit-learn's `drop='first'` parameter. Coefficient values are interpretable only relative to those implicit baselines and are reported here as supporting evidence rather than primary findings.

Regularization strength C (inverse of lambda) is tuned via 5-fold cross-validation.

4.1 Hyperparameter Tuning

In [ ]:
```
pipe_lasso = Pipeline(
    [
        ("preprocessor", preprocessor),
        (
            "classifier",
            LogisticRegression(
                penalty="l1",
                solver="liblinear",  # liblinear is required for L1 penalty
                max_iter=1000,
                random_state=RANDOM_STATE,
            ),
        ),
    ]
)

# C is inverse regularization strength: smaller C = stronger penalty.
# Log-spaced search (10^-3 to 10^2) across 20 values covers a wide range
# without exhaustive computation.
param_grid_lasso = {"classifier__C": np.logspace(-3, 2, 20)}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_lasso = GridSearchCV(
    pipe_lasso,
    param_grid_lasso,
    cv=cv,
    scoring="roc_auc",
    n_jobs=-1,
    verbose=0,
)
grid_lasso.fit(X_train, y_train)

best_C_lasso = grid_lasso.best_params_["classifier__C"]
print(f"Best C (LASSO): {best_C_lasso:.4f}")
print(f"Best CV AUC (LASSO): {grid_lasso.best_score_:.4f}")
```

4.2 Test Set Evaluation

In [ ]:
```
best_lasso = grid_lasso.best_estimator_

y_pred_lasso = best_lasso.predict(X_test)
y_prob_lasso = best_lasso.predict_proba(X_test)[:, 1]

metrics_lasso = {
    "Model": "LASSO Logistic Regression",
    "Accuracy": accuracy_score(y_test, y_pred_lasso),
    "Precision": precision_score(y_test, y_pred_lasso),
    "Recall": recall_score(y_test, y_pred_lasso),
    "F1": f1_score(y_test, y_pred_lasso),
    "ROC AUC": roc_auc_score(y_test, y_prob_lasso),
}

print("LASSO Logistic Regression — Test Set Performance")
for k, v in metrics_lasso.items():
```

```
    if k != "Model":
        print(f"  {k:<12} {v:.4f}")
print(f"\n  SUPPORT benchmark AUC: {SUPPORT_AUC:.4f}")
```

4.3 Coefficient Analysis

In [ ]:
```
# Inspect which features LASSO zeroed out at the best C.
# Zero coefficients = features LASSO deemed uninformative given the regulari
lasso_clf = best_lasso.named_steps["classifier"]
preprocessed_feature_names = (
    best_lasso.named_steps["preprocessor"].get_feature_names_out().tolist()
)

coef_df = pd.DataFrame(
    {"Feature": preprocessed_feature_names, "Coefficient": lasso_clf.coef_[0
).sort_values("Coefficient", key=abs, ascending=False)

n_zeroed = (coef_df["Coefficient"] == 0).sum()
print(f"Features zeroed by LASSO : {n_zeroed} / {len(coef_df)}")
print("\nTop 15 features by absolute coefficient:")
print(coef_df.head(15).to_string(index=False, float_format="{:.4f}".format))
```

In [ ]:
```
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_lasso),
    display_labels=["Survived", "Died"],
).plot(ax=ax, colorbar=False)
ax.set_title("Confusion Matrix — LASSO Logistic Regression", fontweight="bol
plt.tight_layout()
plt.show()
```

Section 5: Random Forest

Random Forest is an ensemble of decision trees trained on bootstrap samples with random feature subsets at each split. It captures nonlinear relationships and feature interactions, including the albumin × disease group and age × disease group interactions identified in [EDA REF], without requiring explicit interaction terms to be specified.

The cost is reduced interpretability relative to logistic regression. Feature importance scores (mean decrease in impurity) partially recover interpretability by ranking predictors by contribution across all trees.

The EDA phase directly addresses clinical factor identification through hypothesis testing and interaction analysis [EDA REF] so the modeling phase is free to prioritize predictive performance over interpretability; this makes Random Forest a justified candidate despite this tradeoff.

5.1 Learning Curve: Number of Trees

Before tuning hyperparameters, we establish an empirical basis for selecting the number of trees (`n_estimators`). The learning curve plots mean cross-validated AUC against a range of tree counts, identifying where performance stabilizes and additional trees yield diminishing returns.

In [ ]:
```
from sklearn.model_selection import cross_val_score

n_estimator_range = [50, 100, 200, 300, 500, 750, 1000]
mean_aucs = []

for n in n_estimator_range:
    rf = Pipeline(
        [
            ("preprocessor", preprocessor),
            (
                "classifier",
                RandomForestClassifier(
                    n_estimators=n, random_state=RANDOM_STATE, n_jobs=-1
                ),
            ),
        ]
    )
    scores = cross_val_score(rf, X_train, y_train, cv=cv, scoring="roc_auc")
    mean_aucs.append(scores.mean())

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(n_estimator_range, mean_aucs, marker="o", linewidth=2)
ax.set_xlabel("n_estimators")
ax.set_ylabel("Mean CV AUC")
ax.set_title("Random Forest — AUC vs. Number of Trees", fontweight="bold")
plt.tight_layout()
plt.show()
```

AUC stabilizes around 500 trees, with negligible gains beyond that point. An n_estimators value of 500 was selected based on this curve; increasing further would add compute cost without meaningful improvement in discrimination.

5.2 Hyperparameter Tuning

In [ ]:
```
pipe_rf = Pipeline(
    [
        ("preprocessor", preprocessor),
        (
            "classifier",
            RandomForestClassifier(
                random_state=RANDOM_STATE,
                n_jobs=-1,
            ),
        ),
```

```
    ]
)

# n_estimators fixed at 500 per learning curve above; stabilization
# threshold identified empirically. Grid search tunes max_depth and
# min_samples_leaf only.
#
# max_depth: None allows full tree growth (unconstrained); 10 and 20 test
# progressively constrained trees to evaluate whether limiting depth
# reduces overfitting without sacrificing discrimination.
#
# min_samples_leaf: controls minimum node size. 1 allows single-sample
# leaves (low bias, high variance); 5 and 10 increase regularization
# by requiring larger leaf populations before a split is accepted.
param_grid_rf = {
    "classifier__n_estimators": [500],
    "classifier__max_depth": [None, 10, 20],
    "classifier__min_samples_leaf": [1, 5, 10],
}

grid_rf = GridSearchCV(
    pipe_rf,
    param_grid_rf,
    cv=cv,
    scoring="roc_auc",
    n_jobs=-1,
    verbose=1,
)
grid_rf.fit(X_train, y_train)

print(f"Best params (RF) : {grid_rf.best_params_}")
print(f"Best CV AUC (RF) : {grid_rf.best_score_:.4f}")
```

5.3 Test Set Evaluation

In [ ]:

```
best_rf = grid_rf.best_estimator_

y_pred_rf = best_rf.predict(X_test)
y_prob_rf = best_rf.predict_proba(X_test)[:, 1]

metrics_rf = {
    "Model": "Random Forest",
    "Accuracy": accuracy_score(y_test, y_pred_rf),
    "Precision": precision_score(y_test, y_pred_rf),
    "Recall": recall_score(y_test, y_pred_rf),
    "F1": f1_score(y_test, y_pred_rf),
    "ROC AUC": roc_auc_score(y_test, y_prob_rf),
}

print("Random Forest: Test Set Performance")
for k, v in metrics_rf.items():
    if k != "Model":
        print(f"  {k:<12} {v:.4f}")
print(f"\n  SUPPORT benchmark AUC: {SUPPORT_AUC:.4f}")
```

5.4 Feature Importances

In [ ]:
```
# Feature importances — mean decrease in impurity across all trees.
# Provides a model-native interpretability proxy; compare top features
# against LASSO coefficients and EDA hypothesis testing rankings.
rf_clf = best_rf.named_steps["classifier"]
importance_df = pd.DataFrame(
    {
        "Feature": preprocessed_feature_names,
        "Importance": rf_clf.feature_importances_,
    }
).sort_values("Importance", ascending=False)

fig, ax = plt.subplots(figsize=(10, 6))
top20 = importance_df.head(20)
ax.barh(top20["Feature"][::-1], top20["Importance"][::-1], edgecolor="white"
ax.set_xlabel("Mean Decrease in Impurity")
ax.set_title("Random Forest — Top 20 Feature Importances", fontweight="bold"
plt.tight_layout()
plt.show()

print("\nTop 15 features by importance:")
print(importance_df.head(15).to_string(index=False))
```

In [ ]:
```
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_rf),
    display_labels=["Survived", "Died"],
).plot(ax=ax, colorbar=False)
ax.set_title("Confusion Matrix — Random Forest", fontweight="bold")
plt.tight_layout()
plt.show()
```

Section 6: ROC Curve Comparison

All three candidate models are evaluated against each other and the SUPPORT benchmark using ROC curves and a summary metrics table. AUC is the primary comparison metric as it is threshold-agnostic and directly comparable to the 0.787 benchmark reported by Knaus et al. (1995).

In [ ]:
```
fig, ax = plt.subplots(figsize=(8, 7))

models = [
    ("Logistic Regression (baseline)", y_prob_lr, "steelblue", "-"),
    ("LASSO Logistic Regression", y_prob_lasso, "darkorange", "-"),
    ("Random Forest", y_prob_rf, "seagreen", "-"),
]

for name, probs, color, ls in models:
```

```
    fpr, tpr, _ = roc_curve(y_test, probs)
    auc = roc_auc_score(y_test, probs)
    ax.plot(
        fpr,
        tpr,
        color=color,
        linestyle=ls,
        linewidth=2,
        label=f"{name} (AUC = {auc:.4f})",
    )

# SUPPORT benchmark — horizontal reference at the reported AUC value
ax.axhline(
    SUPPORT_AUC,
    color="crimson",
    linestyle=":",
    linewidth=1.5,
    label=f"SUPPORT benchmark — Knaus et al. (1995) AUC = {SUPPORT_AUC:.4f}"
)
ax.plot(
    [0, 1],
    [0, 1],
    color="gray",
    linestyle="--",
    linewidth=1,
    label="Random classifier (AUC = 0.50)",
)

ax.set_xlabel("False Positive Rate", fontsize=11)
ax.set_ylabel("True Positive Rate", fontsize=11)
ax.set_title(
    "ROC Curve Comparison — All Models vs. SUPPORT Benchmark",
    fontsize=12,
    fontweight="bold",
)
ax.legend(fontsize=9, loc="lower right")
plt.tight_layout()
plt.show()
```

In [ ]:
```
# Side-by-side metrics table with delta vs. SUPPORT benchmark
all_metrics = pd.DataFrame([metrics_lr, metrics_lasso, metrics_rf]).set_inde
all_metrics["vs. SUPPORT AUC"] = (all_metrics["ROC AUC"] - SUPPORT_AUC).rour

print("Model Comparison Summary:\n")
print(all_metrics.round(4).to_string())
print(f"\nSUPPORT benchmark AUC: {SUPPORT_AUC}  (Knaus et al., 1995)")
```

Section 7: Model Selection & Justification

Three candidate models were trained and evaluated against the SUPPORT prognostic model benchmark (AUC = 0.787; Knaus et al., 1995).

Model	Accuracy	Precision	Recall	F1	ROC AUC	vs. SUPPORT
Logistic Regression (baseline)	0.6864	0.6895	0.6014	0.6425	0.7469	-0.0401
LASSO Logistic Regression	0.6875	0.6914	0.6014	0.6433	0.7469	-0.0401
Random Forest	0.7029	0.7091	0.6202	0.6617	0.7651	-0.0219

SUPPORT Model Comparison

The SUPPORT prognostic model (Knaus et al., 1995) was a Cox proportional hazards model built on five components: primary disease category, a composite physiology score (sps) derived from 11 day-3 lab variables, neurologic function (Glasgow coma scale), patient age, and number of prior hospital days. It was developed on 4,301 phase I patients and validated on 4,028 independent phase II patients, achieving an AUC of 0.78 on the phase II holdout.

Our EDA [EDA REF] confirmed that the SUPPORT model's surv6m survival probability estimates achieve an AUC of 0.787 on the SUPPORT2 dataset (the random sample of phases I and II used in this project) establishing a directly comparable benchmark on the same data our models were trained and tested on.

Our Random Forest model achieves an AUC of 0.7651 on the same held-out test set, falling short of the SUPPORT benchmark by 0.0219. This gap reflects the more constrained conditions under which our models operate compared to the original SUPPORT model:

- `sps` (SUPPORT physiology sub-score) excluded: a direct input to the SUPPORT model, excluded here to avoid circular prediction
- `aps` (APACHE III score) excluded: Knaus et al. evaluated SUPPORT and APACHE III as independent competing systems; including either as a feature in the other has no methodological precedent
- `prg6m`, `prg2m` (physician estimates) excluded: removing the physician-enhanced component that Knaus et al. showed improved SUPPORT AUC to 0.82

The comparison is not perfectly controlled; the SUPPORT model was a survival model predicting continuous time-to-event outcomes while our Random Forest is a binary classifier predicting 180-day mortality. AUC measures discrimination in both cases but the underlying modeling frameworks differ. This limitation should be acknowledged in the technical report.

Selected model: Random Forest

All three models fell below the SUPPORT benchmark on raw AUC. The AUC difference between Random Forest (0.7651) and the logistic models (0.7469) is 0.018, which is within normal sampling variation on a single holdout set and not statistically significant on its own. Model selection therefore rests on scientific grounds rather than raw AUC.

The project's primary objective is identifying which day-3 clinical and demographic factors most strongly inform 180-day mortality. This was addressed in the EDA phase through Mann-Whitney U hypothesis testing [EDA REF], interaction analysis [EDA REF], and disease group stratification [EDA REF]. With interpretability handled upstream, the modeling phase is free to optimize for predictive performance.

Random Forest is selected on three grounds:

1. **Highest AUC**: 0.7651 vs. 0.7469 for LASSO, representing the strongest discrimination between survivors and non-survivors on held-out data.

2. **Interaction capture**: Random Forest implicitly models the albumin × disease group and age × disease group interactions identified in EDA Section 8 without requiring explicit interaction terms. These interactions were a defining feature of the original SUPPORT model (Knaus et al., 1995) and are biologically meaningful.

3. **LASSO redundancy**: Baseline logistic regression and LASSO produced identical metrics across all measures (AUC = 0.7469 for both). This confirms that the feature set is already clean and regularization has no meaningful work to do.

The gap relative to the SUPPORT benchmark is consistent with the exclusion of model-derived inputs that SUPPORT relied upon: sps (SUPPORT physiology sub-score), avtisst (intervention intensity averaged over the full observation period), and sfdm2 (2-month functional disability). The substantial drop in AUC when these variables are excluded suggests that a significant portion of SUPPORT's discriminative ability derived from engineered or post-hoc features rather than raw day-3 clinical measurements alone. Our models operate strictly on the latter, which is a more constrained and methodologically cleaner comparison.

In [ ]:
```
FINAL_MODEL = best_rf
FINAL_MODEL_NAME = "Random Forest"

y_pred_final = FINAL_MODEL.predict(X_test)
y_prob_final = FINAL_MODEL.predict_proba(X_test)[:, 1]
```

```
final_auc = roc_auc_score(y_test, y_prob_final)

print(f"Final Model: {FINAL_MODEL_NAME}")
print(f"  Accuracy     {accuracy_score(y_test, y_pred_final):.4f}")
print(f"  Precision    {precision_score(y_test, y_pred_final):.4f}")
print(f"  Recall       {recall_score(y_test, y_pred_final):.4f}")
print(f"  F1           {f1_score(y_test, y_pred_final):.4f}")
print(
    f"  ROC AUC      {final_auc:.4f}  ({final_auc - SUPPORT_AUC:.4f} vs. SUP
)
```

SUPPORT2 research questions: disease group, day-3 physiology, and the ML questions (Michael)

This notebook covers five questions on the Gold feature set: two inference questions (disease group, day-3 physiology) and three prediction questions (a day-3 classifier, whether it beats the 1990s surv6m score, and whether the same variables drive both the inference and the prediction). Every section ran on Gold. The one exception is ML-Q2, where the legacy surv6m score was read from the Silver table for the comparison, since surv6m is a model output and was not part of the Gold features.

RQ1: disease group and 180-day mortality

In [ ]:
```
# Imports
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from statsmodels.stats.multitest import multipletests

# Constants
RANDOM_STATE = 42
ALPHA = 0.05
REFERENCE_GROUP = "ARF/MOSF w/Sepsis"
np.random.seed(RANDOM_STATE)

# add repo root to sys.path so the utils import resolves whether the
# notebook runs from the repo root or from src/notebooks/
_here = Path.cwd()
_proj_root = _here if (_here / "utils").exists() else (_here / "../..").resc
if str(_proj_root) not in sys.path:
```

```
    sys.path.insert(0, str(_proj_root))
from utils.dataset import load_csv  # noqa: E402
```

RQ1: does a patient's disease group shape their odds of dying within 180 days?

This is the project's first and primary research question, and it sets the baseline the rest of the analysis builds on. The study is organized around disease group, every later model holds it constant, and the disease-group signal is the strongest in the cohort. Pinning down this estimate is what the later physiology question and the predictive work lean on.

As an inference question, the model checks whether a patient's main illness is associated with the odds of death within 180 days, and by how much, after age, sex, and overall severity are held constant. Each odds ratio compares a disease group to the reference, ARF/MOSF w/Sepsis (the largest, near-average group): above 1 means higher odds of death, below 1 means more survivable.

In [ ]:
```
# Load the Gold feature set (model-ready; it now carries sex, so the inferen
gold = load_csv("support2_model_features.csv")
print("rows, cols:", gold.shape)

# Base rate of the outcome (roughly half the cohort died within 180 days).
base_rate = gold["death_180d"].mean()
print(
    f"death_180d base rate: {base_rate:.3f} "
    f"({base_rate * 100:.1f}% died within 180 days)"
)

gold.head()
```

In [ ]:
```
# Columns we actually model: the disease group, the controls, and the target
MODEL_COLS = ["death_180d", "age", "sex", "scoma", "dzgroup"]

# leakage and benchmark columns not in the model
# (picking MODEL_COLS already drops them; this just confirms none slipped th
LEAKAGE_COLS = [
    "surv2m",
    "surv6m",
    "prg2m",
    "prg6m",
    "aps",
    "sps",
    "death",
    "hospdead",
    "d.time",
    "d_time",
    "slos",
    "dnr",
    "dnrday",
    "sfdm2",
```

```
]

model_df = gold[MODEL_COLS].dropna()
print("modeling frame shape:", model_df.shape)

leaked = [c for c in LEAKAGE_COLS if c in model_df.columns]
print("leakage/benchmark columns present:", leaked if leaked else "none (cle

model_df.head()
```

The model

One multivariable logistic regression was fit:

```
death_180d ~ age + sex + scoma + C(dzgroup, reference =
"ARF/MOSF w/Sepsis")
```

- death_180d = died within 180 days (1 / 0), the outcome.
- age = per year, held constant as a confounder.
- sex = a demographic confounder, held constant.
- scoma = the day-3 coma score, the in-model severity control (aps and sps were pulled out as benchmarks, so scoma carries severity here).
- C(dzgroup, ...) = disease group as categories, each compared to the sepsis reference.

Logistic because the outcome is yes/no. Multivariable so the disease-group effect is measured after age, sex, and severity are accounted for, not before.

In [ ]:
```
# Fit the multivariable logistic regression with statsmodels. The reference
# group is fixed so every disease-group odds ratio is read against sepsis.
formula = (
    "death_180d ~ age + sex + scoma + "
    f'C(dzgroup, Treatment(reference="{REFERENCE_GROUP}"))'
)
model = smf.logit(formula, data=model_df).fit(disp=False)
print(model.summary())
```

In [ ]:
```
# Build the odds-ratio table from the fitted model.
params = model.params
conf = model.conf_int()
conf.columns = ["ci_low", "ci_high"]

or_table = pd.DataFrame(
    {
        "OR": np.exp(params),
        "OR_ci_low": np.exp(conf["ci_low"]),
        "OR_ci_high": np.exp(conf["ci_high"]),
        "p_value": model.pvalues,
    }
)
```

```
# keep just the disease-group terms for the FDR step and the forest plot
dz_mask = or_table.index.str.startswith("C(dzgroup")
dz_table = or_table[dz_mask].copy()

# BH-FDR correction for testing several groups at once
dz_table["p_bh"] = multipletests(dz_table["p_value"], alpha=ALPHA, method="f

# pull the group name out of the long statsmodels label
dz_table.index = [name.split("T.")[-1].rstrip("]") for name in dz_table.inde
dz_table.index.name = "disease_group"

# sort by odds ratio, highest odds of death at the top
dz_table = dz_table.sort_values("OR", ascending=False)
dz_table[["OR", "OR_ci_low", "OR_ci_high", "p_value", "p_bh"]].round(4)
```

In [ ]:
```
# the controls (age, sex, scoma), shown next to the disease groups
control_terms = [
    t for t in or_table.index if t in ("age", "scoma") or t.startswith("sex"
]
or_table.loc[control_terms, ["OR", "OR_ci_low", "OR_ci_high", "p_value"]].ro
```

In [ ]:
```
# Labeled forest plot: one dot per disease group with its 95% CI bar.
fig, ax = plt.subplots(figsize=(8, 5))

groups = dz_table.index.tolist()
y = np.arange(len(groups))
or_vals = dz_table["OR"].to_numpy()
low = dz_table["OR_ci_low"].to_numpy()
high = dz_table["OR_ci_high"].to_numpy()

xerr = np.vstack([or_vals - low, high - or_vals])
ax.errorbar(
    or_vals,
    y,
    xerr=xerr,
    fmt="o",
    color="black",
    ecolor="gray",
    capsize=3,
    linestyle="none",
)

ax.axvline(1.0, linestyle="--", color="red", linewidth=1)
ax.set_xscale("log")
ax.set_yticks(y)
ax.set_yticklabels(groups)
ax.invert_yaxis()  # highest odds ratio on top
ax.set_xlabel("Odds ratio of death within 180 days (log scale)")
ax.set_ylabel("Disease group")
ax.set_title(
    "RQ1: disease-group odds of 180-day death\n"
    f"reference group = {REFERENCE_GROUP}; dashed line = no effect (OR = 1)"
)
fig.tight_layout()
plt.show()
```

What the disease groups showed

The disease a patient came in with shaped the odds of dying within 180 days a lot, even after holding age, sex, and severity constant. The highest-odds group was MOSF w/Malig (multi-organ failure with cancer), at about 4.6 times the odds of the sepsis reference group, followed by Lung Cancer at about 2.9 times. Cirrhosis (about 1.6 times) and Colon Cancer (about 1.3 times) were also higher than the reference. On the other end, CHF (heart failure) and COPD were the most survivable, at about 0.54 and 0.61 times the reference odds.

Coma came out at about 1.2 times the reference, but its confidence interval crossed 1 and it did not survive the BH-FDR correction (adjusted $p = 0.12$), so this model cannot tell coma apart from the sepsis reference. That runs against an earlier expectation that coma would come in high.

The two controls behaved as expected. Each extra year of age multiplied the odds by about 1.02 (a 2.3 percent rise per year, $p < 0.001$), and each point of the coma score raised the odds by about 1.03 ($p < 0.001$). Sex made only a small difference: men had about 1.08 times the odds of women, and that gap was not significant ($p = 0.08$) once disease, age, and severity were accounted for. That is why sex is in the model as a control, not as one of the disease-group effects the question is actually about: its own effect is small, but adjusting for it keeps the disease-group odds ratios comparable.

Every disease-group difference except Coma stayed significant after the Benjamini-Hochberg FDR correction, so these are not an artifact of testing several groups at once.

RQ2: day-3 physiology beyond disease group

In [ ]:
```python
# Imports
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor
from patsy import dmatrix
from scipy import stats

# Constants
ALPHA = 0.05
REFERENCE_GROUP = "ARF/MOSF w/Sepsis"
```

```
CORE_PHYS = ["meanbp", "wblc", "crea"]
SENS_PHYS = ["alb", "bili"]  # heavily imputed with normal values, checked s

# add repo root to sys.path so the utils import resolves whether the
# notebook runs from the repo root or from src/notebooks/
_here = Path.cwd()
_proj_root = _here if (_here / "utils").exists() else (_here / "../..").resc
if str(_proj_root) not in sys.path:
    sys.path.insert(0, str(_proj_root))
from utils.dataset import load_csv  # noqa: E402
```

RQ2: does day-3 physiology add signal on top of disease group?

Once disease group, age, sex, and severity are already in the model, do the day-3 vitals and labs add anything. The catch is that a sick patient's numbers often just restate how sick the disease already implies, so the real test is whether physiology carries new signal after disease group is accounted for, not on its own.

In [ ]:
```
gold = load_csv("support2_model_features.csv")
BASE = ["death_180d", "age", "sex", "scoma", "dzgroup"]
model_df = gold[BASE + CORE_PHYS + SENS_PHYS].dropna()
print("rows used:", model_df.shape[0])
print("death_180d base rate:", round(model_df["death_180d"].mean(), 3))
model_df.head()
```

Three nested models

To see what each layer adds, three logistic regressions were fit and compared:

```
M1: death_180d ~ age + sex + scoma
M2: M1 + disease group
M3: M2 + day-3 physiology
```

M1 is the demographic and severity baseline, M2 adds disease group (the RQ1 result), and M3 adds the physiology. Comparing the fit from M1 to M2 to M3 showed how much disease group explained and how much physiology added on top of it. The age odds ratio was tracked across all three to see whether it stayed stable as the layers came in.

In [ ]:
```
f1 = "death_180d ~ age + sex + scoma"
f2 = f1 + f' + C(dzgroup, Treatment(reference="{REFERENCE_GROUP}"))'
f3 = f2 + " + " + " + ".join(CORE_PHYS + SENS_PHYS)

m1 = smf.logit(f1, model_df).fit(disp=False)
m2 = smf.logit(f2, model_df).fit(disp=False)
m3 = smf.logit(f3, model_df).fit(disp=False)
```

```
triple = pd.DataFrame(
    {
        "model": ["age + controls", "+ disease group", "+ physiology"],
        "age_OR": [np.exp(m.params["age"]) for m in (m1, m2, m3)],
        "pseudo_R2": [m.prsquared for m in (m1, m2, m3)],
        "log_likelihood": [m.llf for m in (m1, m2, m3)],
    }
).round({"age_OR": 3, "pseudo_R2": 4, "log_likelihood": 1})
triple
```

In [ ]:
```
# likelihood-ratio test: does physiology add signal beyond disease group (M3
lr_stat = 2 * (m3.llf - m2.llf)
lr_df = len(CORE_PHYS + SENS_PHYS)
lr_p = stats.chi2.sf(lr_stat, lr_df)
print(f"LR test (M3 vs M2): chi2 = {lr_stat:.1f}, df = {lr_df}, p = {lr_p:.2
```

The physiology odds ratios

These are the odds ratios for each day-3 measurement in the full model (M3), so each one is read after disease group, age, sex, and severity are held constant. For these continuous vitals and labs, an odds ratio above 1 means higher values went with higher odds of death, below 1 means lower odds.

In [ ]:
```
ci = m3.conf_int()
phys = CORE_PHYS + SENS_PHYS
phys_table = pd.DataFrame(
    {
        "OR": np.exp(m3.params[phys]),
        "OR_ci_low": np.exp(ci.loc[phys, 0]),
        "OR_ci_high": np.exp(ci.loc[phys, 1]),
        "p_value": m3.pvalues[phys],
    }
).round(4)
phys_table
```

In [ ]:
```
# VIF on the continuous predictors, flag anything above 5 as collinear
X = dmatrix("age + scoma + " + " + ".join(phys), model_df, return_type="data
vif = pd.DataFrame(
    {
        "predictor": [c for c in X.columns if c != "Intercept"],
        "VIF": [
            variance_inflation_factor(X.values, i)
            for i, c in enumerate(X.columns)
            if c != "Intercept"
        ],
    }
).round(2)
vif
```

Sensitivity check on albumin and bilirubin

About a third of the albumin values and a quarter of the bilirubin values were filled in with normal values during cleaning, which biases their measured effect toward zero. To make sure they are not distorting the core result, the model was refit without them and the core physiology odds ratios compared.

In [ ]:
```
f3_core = f2 + " + " + " + ".join(CORE_PHYS)
m3_core = smf.logit(f3_core, model_df).fit(disp=False)
sens_compare = pd.DataFrame(
    {
        "OR_with_alb_bili": [np.exp(m3.params[c]) for c in CORE_PHYS],
        "OR_without": [np.exp(m3_core.params[c]) for c in CORE_PHYS],
    },
    index=CORE_PHYS,
).round(4)
sens_compare
```

What the physiology added

Disease group carried the analysis. Pseudo R-squared went from 0.06 (age, sex, severity) to 0.12 once disease group was added, and only to 0.13 with physiology. The day-3 vitals and labs added only a small slice on top of disease group, though the likelihood-ratio test showed it was real (chi-square 93 on 5 df, p well below 0.001). Age stayed flat across the three models, so it held as a confounder.

Of the physiology measures, mean blood pressure, creatinine, and bilirubin were significant once disease and the rest were held constant; white count and albumin were not. Albumin coming out flat fits its heavy imputation with normal values. No collinearity showed up (all VIFs near 1), and dropping albumin and bilirubin barely moved the core odds ratios. Disease group still did most of the work. Physiology added a small but real increment, mostly from blood pressure, creatinine, and bilirubin.

ML-Q1: predicting 180-day mortality from day-3 data

RQ1 and RQ2 asked which factors are associated with death. ML-Q1 switches from explaining to predicting: given a patient's day-3 record, how well can a model flag who will not survive 180 days. It is judged on a held-out test set the model never saw, by how well it separates survivors from non-survivors (AUROC) and, because the use case is triage, by recall (how many of the patients who died it caught).

The features are the Gold model-ready set with two scrubs. sfdm2, a 2-month functional-status follow-up, and avtisst, the average TISS over days 3 to 25, were both dropped because they are recorded past the day-3 baseline and would leak future information into a day-3 prediction.

In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, confusion_matrix, brier_score_los
from sklearn.preprocessing import StandardScaler
from sklearn.calibration import calibration_curve

gold = load_csv("support2_model_features.csv")
TARGET = "death_180d"
LEAK = [
    "sfdm2",
    "avtisst",
]  # sfdm2 = 2-month follow-up; avtisst = avg TISS over days 3-25, both reac

y = gold[TARGET].to_numpy()
X = pd.get_dummies(
    gold.drop(columns=[TARGET] + LEAK, errors="ignore"), drop_first=True
).astype(float)
row_idx = np.arange(len(gold))

X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(
    X, y, row_idx, test_size=0.30, stratify=y, random_state=42
)
keep = X_train.columns[
    (X_train.std() > 1e-6) & ((X_train != 0).sum() >= 20)
]  # drop near-constant dummies
X_train, X_test = X_train[keep], X_test[keep]
print(
    "train rows:",
    X_train.shape[0],
    " test rows:",
    X_test.shape[0],
    " test deaths:",
    int(y_test.sum()),
)
```

In [ ]:
```
import warnings

scaler = StandardScaler().fit(X_train)
forest = RandomForestClassifier(n_estimators=400, random_state=42, n_jobs=-1
    X_train, y_train
)
p_forest = forest.predict_proba(X_test)[:, 1]

# the logistic fit emits harmless numpy/BLAS matmul RuntimeWarnings on this
# (the feature check found no degenerate column), so they are suppressed for
# only; the metrics below were checked and did not change.
with warnings.catch_warnings():
    warnings.simplefilter("ignore", RuntimeWarning)
```

```
    logit_clf = LogisticRegression(max_iter=5000).fit(
        scaler.transform(X_train), y_train
    )
    p_logit = logit_clf.predict_proba(scaler.transform(X_test))[:, 1]


def scores(name, p):
    pred = (p >= 0.5).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    return {
        "model": name,
        "AUROC": roc_auc_score(y_test, p),
        "recall": tp / (tp + fn),
        "specificity": tn / (tn + fp),
        "Brier": brier_score_loss(y_test, p),
    }


perf = pd.DataFrame(
    [scores("logistic", p_logit), scores("random forest", p_forest)]
).round(3)
perf
```

In [ ]:
```
# confusion matrix for the random forest at the 0.5 threshold
cm = confusion_matrix(y_test, (p_forest >= 0.5).astype(int))
pd.DataFrame(
    cm, index=["actual survived", "actual died"], columns=["pred survived",
)
```

In [ ]:
```
# calibration: do the predicted probabilities match the observed death rates
frac_pos, mean_pred = calibration_curve(y_test, p_forest, n_bins=10)
fig, ax = plt.subplots(figsize=(5, 5))
ax.plot([0, 1], [0, 1], "--", color="gray", label="perfect")
ax.plot(mean_pred, frac_pos, "o-", color="black", label="random forest")
ax.set_xlabel("predicted probability of death")
ax.set_ylabel("observed fraction died")
ax.set_title("ML-Q1: calibration of the random forest")
ax.legend()
fig.tight_layout()
plt.show()
```

How well it predicted

On patients it had never seen, the random forest reached an AUROC of about 0.76 and the logistic about 0.75, above chance but not by a wide margin. Recall sat near 0.64, so the model caught a little under two thirds of the patients who actually died within 180 days. For triage that recall is the number that matters most, since a missed death is the costly error, and it is the first thing to push higher with threshold tuning. The calibration curve tracked the diagonal reasonably.

Two columns were dropped as leakage. With sfdm2 left in, both models scored near 0.90. Dropping sfdm2 brought it to about 0.81, but that still leaned on avtisst, the average TISS over days 3 to 25, which also reaches past the day-3 baseline. Dropping avtisst too gives the honest day-3 result, about 0.76.

ML-Q2: does the model beat the 1990s surv6m score?

SUPPORT2 ships with surv6m, the 180-day survival estimate from the original 1990s SUPPORT model. ML-Q2 races our model against it on the same held-out test patients. surv6m is pulled from Silver and used only as the opponent, never as a feature inside our model, since it is a model output and would leak.

In [ ]:
```
# surv6m is not in Gold (it is a leakage column), so it is read from Silver
# Silver and Gold are the same rows in the same order, so idx_test aligns pc
silver = load_csv("support2_cleaned.csv")
assert len(silver) == len(gold), "silver/gold row count mismatch"
assert (silver["age"].to_numpy() == gold["age"].to_numpy()).all(), (
    "silver/gold row order mismatch"
)

legacy = (
    1.0 - silver.iloc[idx_test]["surv6m"].to_numpy()
)  # old model predicted 180-day death
assert len(legacy) == len(y_test), "legacy/test length mismatch"
auc_legacy = roc_auc_score(y_test, legacy)
auc_forest = roc_auc_score(y_test, p_forest)

fpr_f, tpr_f, _ = roc_curve(y_test, p_forest)
fpr_l, tpr_l, _ = roc_curve(y_test, legacy)
fig, ax = plt.subplots(figsize=(6, 6))
ax.plot(
    fpr_f, tpr_f, color="black", label=f"our random forest (AUROC {auc_fores
)
ax.plot(fpr_l, tpr_l, color="red", label=f"legacy surv6m (AUROC {auc_legacy:
ax.plot([0, 1], [0, 1], "--", color="gray")
ax.set_xlabel("false positive rate")
ax.set_ylabel("true positive rate")
ax.set_title("ML-Q2: our model vs the 1990s surv6m score")
ax.legend(loc="lower right")
fig.tight_layout()
plt.show()
print(f"our random forest AUROC = {auc_forest:.3f}")
print(f"legacy surv6m AUROC     = {auc_legacy:.3f}")
```

Our model versus surv6m

Once the day-3 feature set was clean, the legacy score held up better than our model. The random forest came in at about 0.76 against surv6m's 0.79 on the

same test patients, so the 1990s estimate edged our day-3 model by roughly 0.03 of AUROC. The earlier run that beat surv6m (0.81) was leaning on avtisst, which leaks. So the honest read flips: a model trained only on clean day-3 data did not beat the original SUPPORT estimate, it came in just under it. Matching it within a few points of AUROC from day-3 data alone is still a reasonable result, just not a win, and surv6m remains a strong baseline.

ML-Q3: do the same variables drive inference and prediction?

This ties the two halves together. The inference side (RQ1 and RQ2) ranked variables by how strongly they are associated with mortality (odds ratios). The prediction side ranks them by how much they drive the random forest (feature importance). ML-Q3 lays the two rankings side by side. Where they agree the finding is stronger; where they disagree, that is itself worth a look.

In [ ]:
```
importance = pd.Series(forest.feature_importances_, index=keep).sort_values(
    ascending=False
)
importance.head(10).round(4)
```

In [ ]:
```
# the shared day-3 physiology variables: inference (RQ2) vs prediction (impo
compare_vars = ["meanbp", "wblc", "crea", "alb", "bili"]
bridge = pd.DataFrame(
    {
        "inference_OR": [phys_table.loc[v, "OR"] for v in compare_vars],
        "inference_p": [phys_table.loc[v, "p_value"] for v in compare_vars],
        "rf_importance": [float(importance.get(v, float("nan"))) for v in co
    },
    index=compare_vars,
).round(4)
bridge
```

Where the two views line up

The two rankings agreed only in part. With avtisst removed as a leak, the prediction side spread its importance across age, the ADL score (adlsc), white count, coma score, mean blood pressure, and a broad set of day-3 vitals, rather than leaning on any single dominant variable. The inference side put disease group out front (RQ1), with physiology adding a small increment (RQ2). Both point at overall severity and physiology mattering, but disease group, which carried the strongest inference signal, did not dominate the prediction importance.

Among the shared physiology variables the overlap was loose: some that were significant in the inference model carried little random-forest importance, and one that was not significant ranked higher, so the two lenses do not map one to one. Severity and physiology matter in both views, but no single variable runs the table in both. An odds ratio and a feature importance answer different questions, and that is the reason not to treat them as the same number.

SUPPORT2 Dataset: Bayesian Modeling (Tue)

AAI-500 Applied Statistics for AI | Final Team Project

Notebook Objectives

This notebook covers the **Model Selection & Analysis** phase using a **Bayesian Logistic Regression** implemented in PyMC. For some lab features that are nonlinear, we explore modeling them using B-Spline.

Why Bayesian Logistic Regression?

- LR is simpliest in explainability and quick to run. Most of our features behaves linearly. Some exception are a few lab values which we will address using nonlinear models.
- Unlike regular LR whose coefficients are point estimates, BLR Produces a posterior distribution over each coefficient.
- Provides calibrated uncertainty intervals for every prediction, a necessity for clinical domains.

Section 0: Setup & Imports

In [ ]:
```
import sys
import warnings
from pathlib import Path

import arviz as az
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pymc as pm
import seaborn as sns
from patsy import build_design_matrices, dmatrix
from scipy.special import expit as sigmoid
```

```
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

_here = Path.cwd()
_proj_root = _here if (_here / "utils").exists() else (_here / "../..").resc
if str(_proj_root) not in sys.path:
    sys.path.insert(0, str(_proj_root))

from utils.dataset import load_csv  # noqa: E402
from utils.evaluation import (  # noqa: E402
    auc_table,
    brier_score,
    get_benchmark_preds,
    plot_roc,
)

warnings.filterwarnings("ignore")
pd.set_option("display.float_format", "{:.3f}".format)
sns.set_theme(style="whitegrid")

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

Section 1: Load Data & Feature Selection

We use **12 continuous, 1 binary, and 3 categorical** features (25 model columns after one-hot encoding), based on EDA findings and the feature-selection experiments in Section 1b below:

Feature	Type	EDA justification
`age`	continuous	Non-survivors were ~3 years older on average
`meanbp`	continuous	Hemodynamic instability - direct mortality signal
`bili`	continuous	Liver/sepsis marker; heavy right-skew with a long high-value tail (high IQR-outlier rate in EDA Section 2), linked with liver failure
`bun`	continuous	Kidney function marker
`alb`	continuous	Nutritional/liver status; low in high-mortality groups
`adlsc`	continuous	Functional status - non-survivors more dependent (2.27 vs 1.55)
`scoma`	continuous	SUPPORT coma score - severity of neurological impairment
`pafi`	continuous	PaO2/FiO2 ratio (oxygenation); added in Section 1b AUC search
`hrt`	continuous	Heart rate; added in Section 1b AUC search

| `dzgroup` | categorical | Knaus et al combined dzgroup into dzclass. We first tried dzclass but dzgroup gave us moderate increase in AUC so we deviate from Knaus here. | `ca` | categorical | Cancer status (no/yes/metastatic); added in Section 1b AUC search | | `resp` | continuous | Respiratory rate (day-3 vital sign) | | `temp` | continuous | Body temperature (day-3 vital sign) | | `sod` | continuous | Serum sodium (electrolyte balance) | | `diabetes` | binary | Diabetes comorbidity flag (0/1) | | `income` | categorical | Income bracket (socioeconomic status) |

Section 1b: Feature Selection Experimentation

We use 25 model features (12 continuous + 1 binary + 3 one-hot-encoded categoricals).

TODO: explain feature selection experiments

Handling categorical features

In order for regression models to work on categorical features, we need to one-hot encode them. This prevents strict multicollinearity (dummy variable trap)

- **dzgroup**: We drop the first disease group `ARF/MOSF w/Sepsis` and use it as the baseline reference
- **ca**: We drop metastatic and use it as the baseline cancer status

In [ ]:
```
df = load_csv("support2_cleaned.csv")
print(f"Shape: {df.shape[0]:,} rows  x  {df.shape[1]} columns")

TARGET_COL = "death_180d"
TRAIN_FEATURES = [
    "age",
    "meanbp",
    "bili",
    "bun",
    "alb",
    "adlsc",
    "scoma",
    "pafi",
    "hrt",
    "resp",
    "temp",
    "sod",
]

# Binary comorbidity flags (already 0/1; kept unstandardized like the dummie
BINARY_FEATURES = ["diabetes"]
```

```
# reorganize so we can use cancer=no as baseline which makes the forest plot
df["ca"] = pd.Categorical(df["ca"], categories=["no", "yes", "metastatic"])

## Handling Categorical classes

# One-hot encodings
dzgroup_dummies = pd.get_dummies(df["dzgroup"], prefix="dzgroup", drop_first
ca_dummies = pd.get_dummies(df["ca"], prefix="ca", drop_first=True)
income_dummies = pd.get_dummies(df["income"], prefix="income", drop_first=Tr

DUMMY_COLS = (
    BINARY_FEATURES
    + list(dzgroup_dummies.columns)
    + list(ca_dummies.columns)
    + list(income_dummies.columns)
)
FEATURE_NAMES = TRAIN_FEATURES + DUMMY_COLS

print("dzgroup reference level: ARF/MOSF w/Sepsis")
print("ca reference level: ca_no")
print("income reference level: $11-$25k")
print(f"Dummy/binary columns: {DUMMY_COLS}")
print(f"Total features: {len(FEATURE_NAMES)}")

df_model = pd.concat(
    [
        df[TRAIN_FEATURES + BINARY_FEATURES + [TARGET_COL]],
        dzgroup_dummies,
        ca_dummies,
        income_dummies,
    ],
    axis=1,
)
print("Class balance:")
print(df_model[TARGET_COL].value_counts().rename({0: "Survived", 1: "Died"})
```

Section 2: Preprocessing

Standardizing

To make the continuous features more comparable we employ standardization. Here we rescale our continuous features coefficients using **z-score standardized**:

$$z = \frac{x - \bar{x}}{s}$$

This centers each feature at mean 0 and rescale by a standard deviation of 1.

Data Splitting

Then we do a 80/20 stratified split on the dataset, where 80% is used for training and 20% for testing. We picked 80/20 instead of the better 3-way split with K-fold cross-validation as it's simpler and apt for this small project.

In [ ]:
```
X_continuous = df_model[TRAIN_FEATURES].values
X_categorical = df_model[DUMMY_COLS].values.astype(float)
y = df_model[TARGET_COL].values

# Standardize continuous features only
scaler = StandardScaler()
X_standardized = np.column_stack([scaler.fit_transform(X_continuous), X_cate
n_features = X_standardized.shape[1]

# Stratified 80/20 split
idx_all = np.arange(len(df_model))
X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(
    X_standardized, y, idx_all, test_size=0.2, random_state=RANDOM_SEED, str
)

print(f"Train: {len(y_train):,}  |  Test: {len(y_test):,}")
print(
    f"Train prevalence: {y_train.mean():.1%}  |  Test prevalence: {y_test.me
)

# Descriptive stats of standardized features
feat_stats = pd.DataFrame(X_train, columns=FEATURE_NAMES).describe().T
feat_stats[["mean", "std", "min", "max"]]
```

Section 3: Prior Predictive Check

A prior represent our assumptions about the model's parameters before training. A good prior should produce predicted probabilities spread across [0, 1] rather than collapsing near 0 or 1.

Here we do a quick check to see if our priors are reasonable.

**Histogram shows a nice spread confirming that our features are nicely calibrated **

Prior specification:

- Intercept: `Normal(0, 2)` : gives a wide range, with baseline mortality probabilities between 2% ~ 98%
- Coefficients: `Normal(0, 1)` : penalize one feature from making a large impact. We assume in biology it's implausible for one lab metric to cause death.

In [ ]:
```
with pm.Model() as blr:
    X_data = pm.Data("X_data", X_train)

    # Priors
    intercept = pm.Normal("intercept", mu=0, sigma=2)
    betas = pm.Normal("betas", mu=0, sigma=1, shape=n_features)

    # Linear predictor + sigmoid link
    logit_p = intercept + pm.math.dot(X_data, betas)
    p = pm.Deterministic("p", pm.math.invlogit(logit_p))

    # Likelihood
    obs = pm.Bernoulli("obs", p=p, observed=y_train)

    prior_pred = pm.sample_prior_predictive(draws=200, random_seed=RANDOM_SE

prior_p = prior_pred.prior["p"].values.reshape(-1, len(y_train))
sample_probs = prior_p[:500, 0]  # distribution for one patient

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(sample_probs, bins=40, edgecolor="white", density=True)
ax.set_title(
    "Prior Predictive: Distribution of Predicted Probability (single patient
    fontweight="bold",
)
ax.set_xlabel("Predicted P(death within 180d)")
ax.set_ylabel("Density")
plt.tight_layout()
plt.show()
```

In [ ]:
```
pm.model_to_graphviz(blr)
```

Section 4: Model Training (MCMC - NUTS Sampler)

Sampling is special to Bayesian as it's done to quantify uncertainty. Unlike gradient descent which finds a single best sets of weights, Bayesian samples a thousand sets of weights.

We use the **No-U-Turn Sampler (NUTS)**, a popular gradient-based MCMC (Markov Chain Monte Carlo) algorithm that efficiently explores high-dimensional posterior distributions. It's the default sampler for PyMC.

Sampling parameters:

TODO: tweak and optimize

- `draws=1000` : posterior samples per chain
- `tune=1000` : warm-up/adaptation steps (discarded)

- `chains=4` : independent chains for convergence diagnostics
- `target_accept=0.9` : acceptance rate target (reduces divergences)

In [ ]:
```
with blr:
    trace = pm.sample(
        draws=1000,
        tune=1000,
        chains=4,
        target_accept=0.9,
        random_seed=RANDOM_SEED,
        progressbar=True,
    )
print("Sampling complete.")
```

Section 5: Sanity Check on NUTS Sampler

Key convergence diagnostics:

| Metric | Good threshold | Meaning | Our Results |---|---|---|--| | **R-hat** | < 1.01 | Chains converged to the same distribution | Good | **ESS bulk** | > 400 | Effective sample size - independent posterior draws | Good | **Divergences** | 0 | physics error | Good

In [ ]:
```
summary = az.summary(trace, var_names=["intercept", "betas"])
summary.index = ["intercept"] + FEATURE_NAMES
diag_cols = [c for c in ["mean", "sd", "r_hat", "ess_bulk"] if c in summary.
print(summary[diag_cols].to_string())
print(f"\nDivergences: {trace.sample_stats.diverging.values.sum()}")
```

Section 6: Posterior Coefficient Analysis

The **94% Highest Density Interval (HDI)** is the Bayesian analogue of a confidence interval. In the forest plot below:

- dot: mean of posterior distribution
- line: 94% Highest Density Interval. Effectively saying "We are confident 94% of true weights falls within this line"

Reading the forest plot

- If the HDI **excludes 0**: the feature has a credible effect on mortality
- Positive coefficient: higher value → higher mortality probability
- Negative coefficient: higher value → lower mortality probability

Coefficients are on **standardized** continuous features: a coefficient of 1.0 means a 1-SD increase changes the log-odds by 1.0 (≈ 2.7× odds multiplier).

Results Interpretation

- `scoma` : strongest risk factor
- `ca_yes` , `ca_metastatic` : higher risk relative to the `ca_no` (no-cancer) baseline
- `dzgroup_*` coefficients are relative to the `ARF/MOSF w/Sepsis` baseline group
- `dzgroup_Coma` , `bun` , `alb` : less certain (94% interval near 0)

In [ ]:
```
# Manual forest plot using posterior arrays
post_int_flat = trace.posterior["intercept"].values.flatten()
post_b_flat = trace.posterior["betas"].values.reshape(-1, n_features)

all_names = ["intercept"] + FEATURE_NAMES
means_all = [post_int_flat.mean()] + list(post_b_flat.mean(axis=0))
lower_all = [np.percentile(post_int_flat, 3)] + list(
    np.percentile(post_b_flat, 3, axis=0)
)
upper_all = [np.percentile(post_int_flat, 97)] + list(
    np.percentile(post_b_flat, 97, axis=0)
)

fig, ax = plt.subplots(figsize=(9, 6))
y_pos = range(len(all_names))
ax.errorbar(
    means_all,
    y_pos,
    xerr=[
        np.array(means_all) - np.array(lower_all),
        np.array(upper_all) - np.array(means_all),
    ],
    fmt="o",
    color="steelblue",
    ecolor="steelblue",
    capsize=4,
    markersize=6,
    elinewidth=1.5,
)
ax.axvline(0, color="red", linestyle="--", linewidth=1.2, alpha=0.7)
ax.set_yticks(list(y_pos))
ax.set_yticklabels(all_names)
ax.set_title(
    "Posterior Coefficient Estimates (94% Credible Interval)", fontweight="b
)
ax.set_xlabel("Log-Odds Coefficient")
plt.tight_layout()
plt.show()
```

In [ ]:
```
# Odds ratios from posterior means
post_betas = trace.posterior["betas"].values.reshape(-1, n_features)
```

```
or_mean = np.exp(post_betas.mean(axis=0))
or_lower = np.exp(np.percentile(post_betas, 3, axis=0))
or_upper = np.exp(np.percentile(post_betas, 97, axis=0))

or_df = pd.DataFrame(
    {
        "Feature": FEATURE_NAMES,
        "OR Mean": or_mean.round(3),
        "OR 5.5% ETI": or_lower.round(3),
        "OR 94.5% ETI": or_upper.round(3),
        "Direction": ["Higher risk" if v > 1 else "Lower risk" for v in or_m
    }
).sort_values("OR Mean", ascending=False)

print("Odds Ratios (per 1 SD change for continuous features):")
print(or_df.to_string(index=False))
```

Note on the cancer disease groups: the cancer-related `dzgroup` dummies (`Colon Cancer`, `Lung Cancer`, `MOSF w/Malig`) are collinear with `ca_metastatic` (those patients are all metastatic), so their coefficients are **residual** effects *given* metastatic status — read them together with `ca_metastatic`, not in isolation.

Section 7: Posterior Predictive Check

Now that the model has learned from the data, we ask: *can it reproduce what we observed?* This is a **Posterior Predictive Check (PPC)**. There are two checks we can run, from coarse to refined:

1. **Marginal rate**: does the simulated overall mortality rate match the observed rate? This is necessary but weak: per-patient errors can cancel out in the average.
2. **Calibration (reliability diagram)**: among patients the model assigns ~x% risk, do ~x% actually die?

For brevity we're doing a simple PPC (1).

Result: the simulated overall rate (~46.8%) matches the observed died:survived ratio (4265:4840).

In [ ]:
```
with blr:
    ppc = pm.sample_posterior_predictive(trace, random_seed=RANDOM_SEED)

# Compare simulated vs observed mortality rate
sim_rates = (
    ppc.posterior_predictive["obs"].values.reshape(-1, len(y_train)).mean(ax
)
obs_rate = y_train.mean()
```

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(
    sim_rates,
    bins=40,
    color="steelblue",
    edgecolor="white",
    density=True,
    alpha=0.8,
    label="Simulated rates",
)
ax.axvline(obs_rate, color="red", linewidth=2, label=f"Observed ({obs_rate:.
ax.set_title(
    "Posterior Predictive Check: Simulated vs Observed Mortality Rate",
    fontweight="bold",
)
ax.set_xlabel("Predicted Mortality Rate")
ax.legend()
plt.tight_layout()
plt.show()
```

Section 8: Model Evaluation & Benchmark Comparison

We evaluate the model on the held-out test set (20% of data) using **ROC-AUC**, and compare against the legacy SUPPORT model benchmark (`surv6m`).

Baselines to beat (from EDA Section):

- SUPPORT model (`surv6m`): AUC ≈ 0.789
- Physician estimate (`prg6m`): AUC ≈ 0.756
- APACHE III (`aps`): AUC ≈ 0.658

In [ ]:
```
# Compute posterior predictive probabilities on the test set
post_intercept = trace.posterior["intercept"].values.flatten()
post_betas_all = trace.posterior["betas"].values.reshape(-1, n_features)

# logit_p for each test patient: shape (n_posterior_samples, n_test)
logits_test = post_intercept[:, None] + post_betas_all @ X_test.T
probs_test = sigmoid(logits_test)

pred_mean = probs_test.mean(axis=0)
pred_lower = np.percentile(probs_test, 3, axis=0)
pred_upper = np.percentile(probs_test, 97, axis=0)

# Standard comparison vs the legacy benchmarks (shared helpers in utils.eval
benchmark_preds = get_benchmark_preds(df, idx_test)  # reused by the ROC + S
auc_bayes = roc_auc_score(y_test, pred_mean)
print(
    auc_table(y_test, {"Bayesian LR (linear)": pred_mean}, benchmark_preds).
```

```
        index=False
    )
)
```

In [ ]:
```
plot_roc(
    y_test,
    {"Bayesian LR (linear)": pred_mean, **benchmark_preds},
    title="ROC Curves: Bayesian LR vs Legacy Benchmarks",
)
plt.tight_layout()
plt.show()
```

Section 9: Nonlinear Modeling: B-splines

The main BLR treats every continuous feature **linearly**. But several day-3 vitals are clinically *U-shaped* in mortality (both extremes are dangerous), and the liver marker `bili` rises non-linearly.

Here we refit the model with **cubic B-splines** on the most non-linear features, keep everything else linear, and check whether it improves held-out AUC / calibration over the linear baseline.

9.1 Which features are non-linear?

(Should put in EDA section instead)

For each candidate we split patients into **low** (≤10th pct), **mid** (45–55th pct), and **high** (≥90th pct) by the feature value and compare the observed 180-day mortality in each group:

- **U-shaped**: both low and high mortality exceed the middle (extremes are deadlier than the center).
- **Monotonic**: mortality rises (or falls) steadily from low → high.

We spline the **4 most non-linear** features and leave the rest linear:

- `sod`, `temp`, `resp` — symmetric **U-shapes** (their linear coefficients are ≈ 0 because the two arms cancel).
- `bili` — **monotonic, flat-then-steep** (also heavily right-skewed per EDA); included to test non-linearity beyond U-shapes.

Honorable mentions left linear: `meanbp` and `hrt` are real U's but **one-arm-dominated**, so their linear terms already capture most of the signal. `bun` is essentially flat and `alb` is cleanly monotonic-decreasing.

In [ ]:
```
# Model-free non-linearity screen: observed mortality at low / mid / high de
CANDIDATES = ["sod", "temp", "resp", "bili", "meanbp", "hrt", "bun", "alb"]
rows = []
for f in CANDIDATES:
    x = df_model[f].values
    q = np.quantile(x, [0.10, 0.45, 0.55, 0.90])
    lo = y[x <= q[0]].mean()
    mid = y[(x >= q[1]) & (x <= q[2])].mean()
    hi = y[x >= q[3]].mean()
    if lo > mid and hi > mid:
        shape = "U-shape"
    elif (lo < mid < hi) or (lo > mid > hi):
        shape = "monotonic"
    else:
        shape = "mixed/flat"
    rows.append([f, lo, mid, hi, lo - mid, hi - mid, shape])

nonlin = pd.DataFrame(
    rows, columns=["feature", "lo", "mid", "hi", "lo-mid", "hi-mid", "shape"
)
print(nonlin.to_string(index=False))
```

In [ ]:
```
# 3 U-shaped + bili (monotonic-nonlinear)
SPLINE_FEATURES = ["sod", "temp", "resp", "bili"]

spline_idx = [FEATURE_NAMES.index(f) for f in SPLINE_FEATURES]
keep = [i for i in range(n_features) if i not in spline_idx]
linear_names = [FEATURE_NAMES[i] for i in keep]
X_lin_train = X_train[:, keep]
X_lin_test = X_test[:, keep]

spline_train, spline_test, spline_info = {}, {}, {}
for f in SPLINE_FEATURES:
    xtr = df_model[f].values[idx_train]
    xte = df_model[f].values[idx_test]
    lo_b = float(df_model[f].min())
    hi_b = float(df_model[f].max())
    dm = dmatrix(
        "bs(x, df=5, degree=3, include_intercept=False, "
        "lower_bound=lo_b, upper_bound=hi_b)",
        {"x": xtr, "lo_b": lo_b, "hi_b": hi_b},
        return_type="matrix",
    )
    spline_train[f] = np.asarray(dm)
    spline_info[f] = dm.design_info
    spline_test[f] = np.asarray(build_design_matrices([dm.design_info], {"x"

print(f"Linear features ({len(linear_names)}): {linear_names}")
print(f"Spline features: {SPLINE_FEATURES}")
print(
    "Basis columns per spline feature: "
    f"{[spline_train[f].shape[1] for f in SPLINE_FEATURES]}"
)
```

In [ ]:
```
with pm.Model() as blr_spline:
    intercept = pm.Normal("intercept", mu=0, sigma=2)
    betas_lin = pm.Normal("betas_lin", mu=0, sigma=1, shape=X_lin_train.shap
    eta = intercept + pm.math.dot(X_lin_train, betas_lin)

    # One coefficient vector per spline feature; Normal(0, 1) acts as a mild
    # smoothness prior (a random-walk prior would enforce more smoothness if
    for f in SPLINE_FEATURES:
        coef = pm.Normal(f"spline_{f}", mu=0, sigma=1, shape=spline_train[f]
        eta = eta + pm.math.dot(spline_train[f], coef)

    p = pm.Deterministic("p", pm.math.invlogit(eta))
    pm.Bernoulli("obs", p=p, observed=y_train)

    trace_spline = pm.sample(
        draws=1000,
        tune=1000,
        chains=4,
        target_accept=0.9,
        random_seed=RANDOM_SEED,
        progressbar=True,
    )

# Convergence sanity check (same thresholds as Section 5)
sp_vars = ["intercept", "betas_lin"] + [f"spline_{f}" for f in SPLINE_FEATUR
sp_summary = az.summary(trace_spline, var_names=sp_vars)
print(
    f"Max r_hat: {float(sp_summary['r_hat'].max()):.3f}  |  "
    f"Min ess_bulk: {float(sp_summary['ess_bulk'].min()):.0f}  |  "
    f"Divergences: {int(trace_spline.sample_stats.diverging.values.sum())}"
)
```

In [ ]:
```
pm.model_to_graphviz(blr)
```

In [ ]:
```
# Reconstruct held-out test probabilities from the spline-model posterior
# (same manual approach as Section 8).
sp_intercept = trace_spline.posterior["intercept"].values.flatten()
sp_betas = trace_spline.posterior["betas_lin"].values.reshape(-1, X_lin_test

logits_sp = sp_intercept[:, None] + sp_betas @ X_lin_test.T
for f in SPLINE_FEATURES:
    c = trace_spline.posterior[f"spline_{f}"].values.reshape(
        -1, spline_test[f].shape[1]
    )
    logits_sp = logits_sp + c @ spline_test[f].T
probs_sp_test = sigmoid(logits_sp)
pred_sp_mean = probs_sp_test.mean(axis=0)

# Standard comparison via the shared helpers (linear baseline + spline + ber
auc_spline = roc_auc_score(y_test, pred_sp_mean)
model_preds = {
    "Bayesian LR (linear)": pred_mean,
    "Bayesian LR (spline)": pred_sp_mean,
}
print(auc_table(y_test, model_preds, benchmark_preds).to_string(index=False)
```

```
print(
    f"\nBrier  linear={brier_score(y_test, pred_mean):.3f}  "
    f"spline={brier_score(y_test, pred_sp_mean):.3f}"
)

plot_roc(
    y_test,
    {**model_preds, **benchmark_preds},
    title="ROC: Linear vs B-spline vs Benchmarks",
)
plt.tight_layout()
plt.show()
```

In [ ]:
```
# Partial-effect plots: the centered log-odds contribution of each spline fe
# with its 94% credible band. This is where the U / non-linear shapes become
fig, axes = plt.subplots(1, len(SPLINE_FEATURES), figsize=(4 * len(SPLINE_FE
for ax, f in zip(axes, SPLINE_FEATURES):
    xtr = df_model[f].values[idx_train]
    grid = np.linspace(np.percentile(xtr, 1), np.percentile(xtr, 99), 100)
    Bg = np.asarray(build_design_matrices([spline_info[f]], {"x": grid})[0])
    c = trace_spline.posterior[f"spline_{f}"].values.reshape(-1, Bg.shape[1]

    contrib = Bg @ c.T  # (grid, draws)
    contrib = contrib - contrib.mean(axis=0, keepdims=True)  # center for re
    mean = contrib.mean(axis=1)
    lo = np.percentile(contrib, 3, axis=1)
    hi = np.percentile(contrib, 97, axis=1)

    ax.fill_between(grid, lo, hi, color="steelblue", alpha=0.25)
    ax.plot(grid, mean, color="steelblue", linewidth=2)
    ax.axhline(0, color="grey", linestyle="--", linewidth=0.8)
    # rug of the training values to show where the data (and the curve) is t
    ax.plot(xtr, np.full_like(xtr, lo.min()), "|", color="grey", alpha=0.05)
    ax.set_title(f, fontweight="bold")
    ax.set_xlabel(f"{f} (raw)")
axes[0].set_ylabel("Centered log-odds contribution")
fig.suptitle(
    "B-spline partial effects (U-shapes for sod/temp/resp; flat-then-steep f
    fontweight="bold",
)
plt.tight_layout()
plt.show()
```

9.2 Interpretation

Spline Shapes: the partial-effect curves match the 9.1 screen, `sod`, `temp`, and `resp` bend into the expected **U** (risk lowest near the physiological mid-range and rising at both extremes), while `bili` is **flat at low values then climbs**, the monotonic-nonlinear shape a single linear coefficient could never express.

Gain is modest: AUC moved **0.750 → 0.753 (+0.003)** and Brier **0.201 → 0.200 (−0.001)**. This is expected as these four features have small marginal

effects, so even modeling their shape correctly nudges overall discrimination only slightly.

Takeaway: Nonlinear modeling is inefficient for the current dataset and not worth the increase cost and complexity.

Scope caveat: `meanbp` / `hrt` were deliberately left linear (one-arm-dominated U's whose linear terms already capture most of the signal).